

SAPIENZA
UNIVERSITÀ DI ROMA

DEPARTMENT OF COMPUTER SCIENCE

Title

ENFORCING PATH VALIDATION WITH P4: A CONTROL APPLICATION FOR PROOF OF TRANSIT

Professor:
Marco Polverini

Student:
Mojtaba Taj

Academic Year 2024/2025

Acknowledgment

I have the utmost gratitude for my parents and family, who have been my biggest source of strength along this journey with their unfailing love, support, and encouragement. Their faith in me, sacrifices, and unwavering encouragement have been essential to finishing my thesis.

I would like to express my sincere gratitude to my supervisor, Professor Marco Polverini, for his invaluable guidance, insightful advice, and continuous support throughout the course of this research. His expertise, patience, and encouragement have been essential in helping me navigate challenges and refine my ideas, and I truly appreciate his mentorship.

A special thanks to my uncle Mohammad Zaman Rajabi, whose guidance and encouragement have been a source of inspiration. His support has been invaluable, both spiritually and personally, and I am truly appreciative of his presence in my life.

Completing this thesis has been a challenging yet rewarding experience, and I could not have done it without the support of those who stood by me. To everyone who has played a role, whether big or small, in shaping this journey, thank you.

Abstract

One of the most important challenges in contemporary programmable networks is ensuring secure and efficient network path selection and verification. In large-scale and dynamic networking environments, unauthorized path deviations, misconfigurations, or malicious attacks can compromise data integrity, violate compliance policies, and disrupt network operations. This thesis explores a novel Proof-of-Transit (PoT) verification mechanism designed to address these security concerns in path-aware programmable networks. The proposed approach enables policy-driven path selection at the network edge and simultaneously verifying packet transit without introducing additional probe traffic. A key innovation of this method is the integration of in-situ telemetry for real-time packet traversal tracking and a unique route identifier (routeID) for secure path encoding and validation. To achieve lightweight and scalable path verification, this thesis leverages Shamir's Secret Sharing (SSS), which ensures cryptographic proof of packet transit while minimizing overhead and eliminating per-flow state storage at network nodes. An implementation using the P4 programming language, combined with P4Runtime control for dynamic table management, was developed and tested in an emulated network environment to evaluate the feasibility and effectiveness of the proposed method. Experimental results show that, the method guarantees safe, dependable, and policy-compliant network operations, especially in large-scale fat-tree and data center structures, while maintaining low protocol overhead and significantly reducing forwarding state complexity. The outcomes also demonstrate the strategy's efficiency and scalability, which makes it a workable and reliable solution for programmable network designs and software-defined networking (SDN).

Contents

Acknowledgment **1**

Abstract **2**

1 Introduction **5**
1.1 Traditional Internet Architecture 5
1.2 Evolution Towards Programmable Networking 6
1.3 Proof-of-Transit (PoT) and PolKA in Secure Routing 7

2 Methodology **8**
2.1 Proof of Transit 8
2.2 P4: Programming Protocol-Independent Packet Processors 8
2.2.1 Packet Processing Pipeline 8
2.2.2 P4 Header Definitions 8
2.2.3 Parser Section 9
2.2.4 Match-Action Tables 10
2.2.5 Ingress Processing 11
2.2.6 Egress Processing 13
2.2.7 Deparser Section 14
2.2.8 Switch 15
2.3 P4Runtime 17
2.3.1 P4 Files 18
2.3.2 P4.json 18
2.3.3 P4Info.txt 18
2.3.4 Setting Forwarding Rules 20
2.3.5 Monitoring Network Traffic 20
2.4 PolKA Source Routing 21
2.4.1 Calculating routeID 23
2.4.2 CRC for Implementing mod Operation in P4 25
2.5 Shamir's Secret Sharing 27
2.5.1 Mersenne Numbers 28
2.5.2 Structure 28
2.5.3 Reconstruction by Lagrange Polynomial Interpolation 28
2.5.4 Secret Reconstruction Example 30
2.6 Compose the Onion 32
2.6.1 How Onion Composition Works in Secret Sharing 32
2.6.2 Mathematical Formulation of Onion Composition 32
2.6.3 Example of Onion Secret Sharing 33

3 Deployment **35**
3.1 Seamless Path Migration and Dynamic Routing 35
3.2 System Setup and Implementation 35
3.2.1 Mininet Environment 36
3.2.2 Polynomial-based Path Verification in PoT-PolKA 37
3.2.3 Data Plane Configuration 38
3.2.4 Path Verification at the Egress Node 38
3.3 Linear Topology 39
3.4 P4Runtime Deployment 40
3.4.1 P4Runtime Architecture and Setup 40
3.4.2 Switch Connection Management 40
3.4.3 Installing P4 Programs 42
3.4.4 Table Entry Management 42
3.4.5 IPv4 Forwarding Rule 43
3.4.6 Source Routing Forwarding in P4Runtime 43
3.4.7 Core Switch Configuration 45
3.4.8 NodeID Distribution 46
3.4.9 Reading Table Entries and Monitoring 48
3.4.10 Error Handling and Connection Management 49
3.4.11 Summary of P4Runtime Deployment 49
3.5 Define Parameters for Secret Sharing 51
3.5.1 Poly1: Private Polynomial for Secret Sharing 51
3.5.2 Poly2: Public Polynomial for Metadata Updates 53
3.5.3 CML Calculation 54

4 Proof of Concept **56**
4.1 Verification Phase 56
4.2 Deviation Phase 58

5 Conclusions **60**

References **61**

1 Introduction

The increasing complexity of modern networks and the growing demand for security and efficiency have highlighted the limitations of traditional routing architectures.

1.1 Traditional Internet Architecture

The Internet was designed with a destination-based forwarding model, where packets are forwarded by routers based on the destination IP address. Each router independently decides how to forward a packet using its local routing table, which maps destination addresses to next-hop routers. This architecture relies on dynamic routing protocols, such as OSPF (Open Shortest Path First), RIP (Routing Information Protocol), and BGP (Border Gateway Protocol), to distribute routing information and adapt paths to network changes. While this decentralized approach ensures scalability and adaptability, it also comes with significant limitations. Since routers make independent decisions without knowledge of the entire end-to-end path, there is no inherent path control or verification. This lack of transparency makes traditional routing vulnerable to multiple security risks:

- **Traffic deviation and hijacking**: Malicious actors or misconfigured routers can redirect packets through unintended paths, violating security policies and increasing the risk of attacks.
- **Man-in-the-middle attacks**: Attackers can intercept and manipulate traffic by altering routing tables or exploiting protocol vulnerabilities.
- **Regulatory compliance issues**: Certain industries require strict routing policies, ensuring that data flows through specific geographic locations or security appliances, which traditional routing cannot guarantee.
- **Forwarding Information Base (FIB) corruption**: Errors in routing table updates can cause misrouting, leading to packet loss or degraded performance.

1.2 Evolution Towards Programmable Networking

To overcome these limitations, networking technologies have evolved to incorporate programmability and verification mechanisms. Programmable networking introduces a shift from static, predefined forwarding behaviors to flexible, software-defined approaches that enable network operators to adapt to new security, performance, and compliance requirements.

One of the major advancements in programmable networking is the separation of the control and data planes. Traditionally, network devices contained both control and forwarding logic, limiting flexibility and making policy enforcement complex. In contrast, programmable networking architectures, such as Software-Defined Networking (SDN), decouple these functions, allowing centralized controllers to dictate network behavior dynamically. This enables better traffic engineering, security enforcement, and real-time network optimization. Additionally, data plane programmability has emerged as a crucial innovation.

Technologies like P4 (Programming Protocol-independent Packet Processors) allow network devices to be reprogrammed to handle new protocols, enforce security rules, and process packets in ways that traditional routers and switches cannot. Unlike fixed-function hardware, programmable switches using P4 enable rapid adaptation to changing network demands, making it possible to implement fine-grained traffic control, deep packet inspection, and dynamic policy enforcement without requiring hardware upgrades.

Another key benefit of programmable networking is enhanced security and monitoring. Traditional networks often struggle with blind spots and limited visibility into traffic flows. Programmable data planes address these issues by enabling real-time telemetry, anomaly detection, and automated responses to security threats. This level of granularity helps mitigate attacks such as Distributed Denial-of-Service (DDoS), unauthorized traffic redirection, and policy violations.

In addition, network automation and intent-based networking are gaining traction as organizations seek to reduce operational complexity. With programmable networking, administrators can define high-level policies and let the network autonomously adjust configurations to meet these objectives. This automation reduces human intervention, minimizes errors, and improves the resilience of modern networks.

As networks continue to evolve, the adoption of programmable architectures is becoming essential for ensuring agility, security, and compliance. These innovations pave the way for more robust routing mechanisms, including Proof-of-Transit (PoT) and other verification techniques, ensuring that data flows through designated paths with verifiable security guarantees.

1.3 Proof-of-Transit (PoT) and PolKA in Secure Routing

Proof-of-Transit (PoT) techniques have drawn a lot of interest in the field of secure routing, as they allow packets to be verified as having traveled the correct routes. Through cryptographic algorithms and in-network telemetry, PoT reduces the risk of unauthorized path changes and ensures that predetermined routing policies are followed. The utility of PoT has been further enhanced with new techniques such as secret-sharing and the Polynomial Key-based Architecture (PolKA) that make verification processes in programmable networks lighter and more scalable.

This study investigates a secure routing approach that uses P4 and P4 Runtime, by combining Proof-of-Transit (PoT) principles to enhance path verification and enforce policies. The proposed solution takes advantage of P4's programmability to create custom packet processing logic that embeds path metadata, ensuring that each packet follows a verified route. Additionally, the P4Runtime controller dynamically configures and monitors the forwarding paths. The main goals of this research are to design a scalable, PoT-based secure routing framework, test its performance in a programmable network environment, and demonstrate its ability to reduce common network security risks.

2 Methodology

2.1 Proof of Transit

Proof of Transit (PoT) [1] is a network verification mechanism designed to cryptographically prove that packets have traversed a specific sequence of network nodes. This mechanism is particularly crucial in modern programmable networks, where path verification and security compliance are essential requirements.

2.2 P4: Programming Protocol-Independent Packet Processors

The P4 programming language [2] is a domain-specific high-level language designed to program packet processing pipelines in network devices such as switches, routers, and network interface cards. The primary goal of P4 is to allow network operators and researchers to customize how packets are processed and forwarded in the data plane of a network device.

2.2.1 Packet Processing Pipeline

This is a core concept in P4 programming and defines how a network device (such as a switch or router) processes incoming packets from the moment they arrive until they are forwarded, dropped, or processed further. It is essentially a sequence of stages or steps through which packets are handled, with each stage performing a specific operation.

2.2.2 P4 Header Definitions

The structure of the packet headers that will be processed in the pipeline is specified in the header definition section of a P4 application. By defining the format of the network protocols, the P4 switch can extract, modify, and forward packets in a suitable manner. Typically, this part contains a header stack, metadata structures, header definitions, and type aliases. Type aliases guarantee uniformity and clarity by defining fixed-width data types, such as MAC and IP addresses. Each protocol's fields are specified using the header keyword in header declarations. For specific packet processing, custom headers can be added; however, standard headers such as Ethernet and IPv4 are frequently established. Because each field in a header has a set bit width, packet processing and change are done precisely. Additional information that is utilized during processing but is not included in the original packet is contained in the metadata structure, which is constructed using the struct keyword. Values such as flags, exit port numbers, and temporary variables needed for pipeline decision making can be found in metadata fields. The header definition section plays a crucial role in P4 programming,

enabling the switch to understand packet formats, extract important fields, change header values, and implement custom protocols for more complex networking features.

```
typedef bit<Number>  Name;

header ethernet_t {
    macAddr_t dstAddr;
    macAddr_t srcAddr;
    bit<16>   etherType;
}

header ipv4_t {
    bit<4>    version;
    bit<4>    ihl;
    bit<8>    diffserv;
    bit<16>   totalLen;
    bit<16>   identification;
    bit<3>    flags;
    bit<13>   fragOffset;
    bit<8>    ttl;
    bit<8>    protocol;
    bit<16>   hdrChecksum;
    ip4Addr_t srcAddr;
    ip4Addr_t dstAddr;
}


header potPolka_t {
    bit<160>   routeId;
    bit<16>    rnd;
    bit<16>    cml;
    bit<16>    secret;
}

struct metadata {
}

struct headers {
    ethernet_t  ethernet;
    ipv4_t      ipv4;
    potPolka_t  potPolka;
}
```

Figure 1: P4 Header Definitions

Figure 1 demonstrates how standard headers like Ethernet and IPv4, as well as custom headers such as `potPolka_t`, are defined in P4.

2.2.3 Parser Section

After the header definitions, the first stage of the packet processing pipeline is the parser, which is responsible for analyzing incoming packets and extracting relevant information from their headers. The parser breaks down the packet into its components (e.g., Ethernet and IP headers) based on predefined header formats, which can be

customized in P4. It uses the header types defined in P4 to identify and extract necessary fields, such as source and destination IP addresses, from the packet.

```
parser MyParser(packet_in packet, out headers_t hdr, out metadata_t meta) {
    state start {
        transition parse_ethernet;
    }

    state parse_ethernet {
        packet.extract(hdr.ethernet);
        transition select(hdr.ethernet.etherType) {
            TYPE_IPV4: parse_ipv4;
            TYPE_SRCROUTING: parse_srcRouting;
            default: accept;
        }
    }

    state parse_srcRouting {
        packet.extract(hdr.potPolka);
        transition accept;
    }

    state parse_ipv4 {
        packet.extract(hdr.ipv4);
        transition accept;
    }

}
```

Figure 2: P4 Parser Implementation

Figure 2 shows a P4 parser implementation that extracts Ethernet, IPv4, and source routing headers from incoming packets. The parser transitions between states based on header values, demonstrating how protocol-specific processing is handled dynamically.

2.2.4 Match-Action Tables

After parsing, the packet proceeds through match-action tables, where the network device determines how to handle packets based on their contents. These tables examine specific fields, such as destination IP addresses or source MAC addresses, and apply corresponding actions—like forwarding to a port, modifying headers, or dropping packets.

P4 programs define these tables by specifying key components such as match fields, match types, supported actions, default behaviors, and other table properties. The language supports several match types to provide precise packet processing control: exact matches require an exact field value; Longest Prefix Match (LPM) selects the most specific network prefix; range matches test whether values fall within a range; and ternary matches use value-mask pairs to support wildcards, allowing certain bits to be ignored.

Tables are populated at runtime through control plane interfaces like P4Runtime, enabling dynamic policy updates without restarting devices—a critical feature for responsive Software-Defined Networking environments. This runtime programmability allows network operators to update forwarding behavior, security policies, and traffic management rules on demand. Common applications include forwarding based on destination addresses, Access Control Lists (ACLs) for security, Quality of Service (QoS) for traffic prioritization, load balancing, and Network Address Translation (NAT). The abstraction between match-action logic and hardware enables consistent, portable behavior across diverse devices.

```
action add_sourcerouting_header (port, dmac, routeIdPacket, rnd, cml, secret)
{
        standard_metadata.egress_spec = port;
        hdr.ethernet.dstAddr = dmac;
        hdr.potPolka.setValid();
        log_msg("Ingress: POT-POLKA is valid");
        hdr.potPolka.routeId = routeIdPacket;
        hdr.potPolka.rnd = rnd;
        hdr.potPolka.cml = cml;
        hdr.potPolka.secret = secret;
        hdr.ethernet.etherType = TYPE_SRCROUTING;
}

table tunnel_encap_process_sr {
    key = {
        hdr.ipv4.dstAddr: lpm;
    }
    actions = {
        add_sourcerouting_header;
        drop;
    }
    size = 1024;
    default_action = drop();
}
```

Figure 3: P4 Table Definition for Source Routing Forwarding

Figure 3 shows the P4 table definition for source routing forwarding. The code defines a table named `tunnel_encap_process_sr` that uses the destination IPv4 address as a lookup key with longest prefix matching (lpm). The table can execute either the `add_sourcerouting_header` action to encapsulate packets with source routing information or drop packets if no matching rule is found. With a capacity of 1024 entries and a default drop action, this table represents the core data plane logic for implementing source routing in P4-programmable networks.

2.2.5 Ingress Processing

Once a packet has been parsed and evaluated by match-action tables, it proceeds through the ingress processing stage. Here, more complex actions, such as modifying packet headers or performing a deeper packet inspection, can be performed. This part

of the pipeline is responsible for making final decisions about how the packet is treated before it is sent on its way, either to another device or to a higher layer.
In P4, ingress processing represents a critical control point where the packet's fate is determined based on the results of match-action table lookups and other stateful operations. The ingress control block typically contains a sequence of table applications and conditional logic that implements the forwarding policy of the device.
During ingress processing, the P4 program can:

- **Apply multiple tables in sequence**: Results from one table can influence subsequent table lookups, creating a processing chain that implements complex forwarding logic.
- **Maintain and update stateful elements**: Counters, meters, and registers can track traffic statistics, rate-limit flows, or store information that persists across packets.
- **Perform header modifications**: Fields can be rewritten, headers can be added or removed, and checksums can be updated to implement network functions like NAT, tunneling, or load balancing.
- **Make forwarding decisions**: The output port (or multicast group) for the packet is determined, along with priority or queue information for traffic management.
- **Clone or recirculate packets**: For monitoring or more complex processing, packets can be cloned to the egress pipeline or recirculated back through the ingress pipeline.

The ingress processing stage often includes correlation of information across multiple tables. For example, a first table might classify the packet based on its headers, a second table might apply access control rules based on that classification, and a third table might determine the output port based on both the classification and security decision. This correlation enables sophisticated packet processing that adapts to network conditions and policy requirements.
The results of ingress processing are stored in metadata fields that accompany the packet through the rest of the pipeline. These metadata fields carry important information such as the egress port, traffic class, or custom processing flags that influence subsequent stages of packet processing.

```
control MyIngress(inout headers hdr, inout metadata meta,
                  inout standard_metadata_t standard_metadata) {
    action drop() {
        mark_to_drop(standard_metadata);
    }

    action ipv4_forward(macAddr_t dstAddr, egressSpec_t port) {
        standard_metadata.egress_spec = port;
        hdr.ethernet.srcAddr = hdr.ethernet.dstAddr;
        hdr.ethernet.dstAddr = dstAddr;
        hdr.ipv4.ttl = hdr.ipv4.ttl - 1;
    }

    table ipv4_lpm {
        key = {
            hdr.ipv4.dstAddr: lpm;
        }
        actions = {
            ipv4_forward;
            drop;
            NoAction;
        }
        size = 1024;
        default_action = drop();
    }

    apply {
        {
            ipv4_lpm.apply();
        }
    }
}
```

Figure 4: P4 Ingress Control Implementation

Figure 4 illustrates an ingress control block implemented in P4. It defines two key actions: `drop()`, which marks the packet to be discarded, and `ipv4_forward()`, which updates Ethernet addresses, decrements the IPv4 TTL, and sets the egress port. The table `ipv4_lpm` performs a longest prefix match on the destination IPv4 address and determines whether to forward, drop, or leave the packet unchanged via the `NoAction` directive. The default behavior is to drop unmatched packets. Finally, the `apply` block triggers the table logic for each packet during ingress processing.

2.2.6 Egress Processing

After a packet completes ingress processing, it enters the egress stage, where final actions are taken before it is transmitted out of the device. The egress pipeline provides an additional opportunity to modify packet headers, adjust metadata, or apply network-specific policies based on the results of ingress processing.
The egress stage in P4 enables several important operations, including:

- **Final Header Modifications**: Modify or add final header fields such as VLAN tags, MPLS labels, or telemetry data for network monitoring and diagnostics.

- **Traffic Shaping and Prioritization**: Assigning packets to specific queues or setting priority levels based on Quality of Service (QoS) policies.
- **Security Enforcement**: Adding security-related headers or modifying packets according to compliance requirements before they exit the device.
- **Multicast and Replication Handling**: Managing packets that need to be replicated and sent out to multiple destinations.

The egress control block operates similarly to the ingress block, but focuses on post-ingress actions. Metadata generated during ingress (such as egress port selection, traffic class, or special handling flags) is used to guide egress processing.
While egress processing is often simpler than ingress processing, it is critical for finalizing packet treatments that ensure compliance with network policies, service level agreements, and security standards. In some cases, particularly in programmable switches like Intel Tofino, the egress pipeline can perform deep and complex transformations, enhancing telemetry reporting or modifying packet contents based on dynamic conditions.

```
control MyEgress(inout headers hdr,
                 inout metadata meta,
                 inout standard_metadata_t standard_metadata) {
    apply {
        // Example: perform final packet adjustments here
    }
}
```

Figure 5: P4 Egress Control Implementation

Note on Egress Processing: Egress processing complements ingress logic by ensuring that any required final modifications, tagging, shaping, or security enhancements are properly applied before packet transmission. It plays an essential role in maintaining the correctness, efficiency, and policy compliance of the network.

2.2.7 Deparser Section

The deparser in P4 serves as the final stage of the packet processing pipeline and plays a vital role in reassembling the packet for transmission. Conceptually, the deparser is the reverse of the parser: while the parser extracts headers and fields from the incoming packet, the deparser serializes and emits these headers back into a well-formed packet. Correct header emission order is crucial during deparsing. Headers must be serialized in a sequence that matches the expected protocol stack (e.g., Ethernet before IPv4), ensuring proper interpretation by downstream devices.
In the deparser, only those headers that have been marked as valid during earlier processing stages are emitted. This selective emission allows flexible packet transformation, including header insertion, removal, or modification.

In P4, the deparser is typically implemented as a simple control block that calls the `packet.emit()` function for each valid header in the appropriate order.

```
control MyDeparser(packet_out packet, in headers hdr) {
    apply {
        packet.emit(hdr.ethernet);
        packet.emit(hdr.ipv4);
        packet.emit(hdr.potPolka);
    }
}
```

Figure 6: P4 Deparser Implementation

2.2.8 Switch

The V1Model architecture [3] forms the core foundation for P4 program execution, defining a standardized packet processing pipeline. This architecture consists of six modular stages that operate in sequence to process network traffic. The `V1Switch` declaration binds these components into a unified processing flow, and declaring it as ‘main‘ designates it as the primary architecture for the P4 program. This modular approach enables network engineers to implement complex processing logic while maintaining a clear separation of concerns.

To execute P4 programs targeting the V1Model architecture, developers typically use `BMv2` (Behavioral Model version 2) [4]—a software switch that serves as a reference implementation for P4. The `simple_switch` component of BMv2 is specifically designed to run programs built on V1Model, providing a controlled environment for testing and debugging without requiring specialized hardware. As a result, the V1Model architecture has become the industry standard for P4 programming, supporting everything from basic packet forwarding to advanced traffic engineering solutions.

```
V1Switch(
    MyParser(),
    MyVerifyChecksum(),
    MyIngress(),
    MyEgress(),
    MyDeparser()
) main;
```

Figure 7: P4 Switch

Figure 7: P4 code snippet showing instantiation of the V1Switch architecture with user-defined parser, checksum verification, ingress, egress, and deparser blocks.

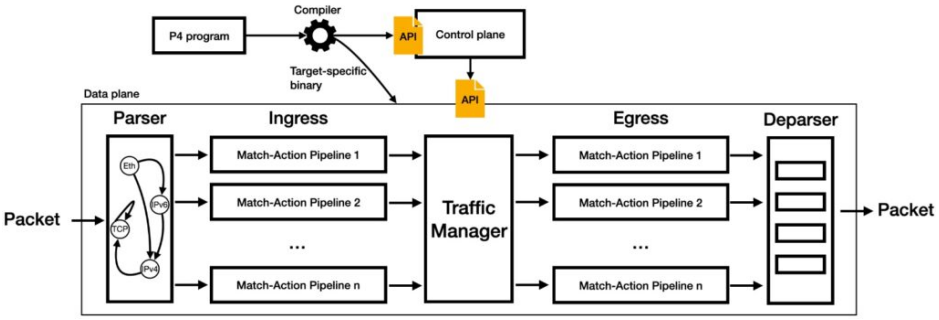


Figure 8: P4 Packet Processing Pipeline

Figure 8: Visual overview of the P4 packet processing pipeline, illustrating the logical stages from parsing to deparsing.

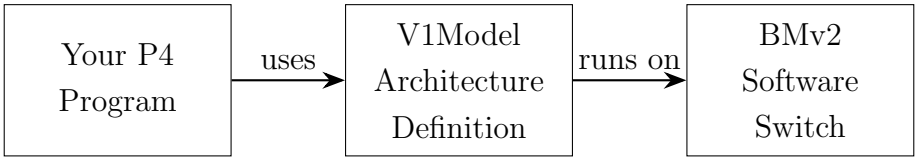


Figure 9: Relationship between a P4 program, the V1Model architecture definition, and the BMv2 software switch used for testing and validation.

2.3 P4Runtime

P4Runtime [5] is an API that allows external control of network devices on which P4 programs are running. It serves as an interface between the control plane and the data plane, enabling external controllers or Software-Defined Networking (SDN) applications to modify data plane behavior without requiring recompilation or reloading of the P4 program. This flexibility is essential in modern network architectures, where real-time updates to forwarding rules, security policies, and traffic management are necessary.

The primary role of P4Runtime is to provide a mechanism for the control plane to manage match-action tables, counters, meters, and other components of the P4-defined packet processing pipeline. Unlike traditional static configurations, where forwarding rules and security policies are predefined at compile time, P4Runtime allows these elements to be modified dynamically.

P4Runtime operates using gRPC (Google Remote Procedure Call), a widely-used high-performance communication protocol that enables secure and efficient interaction between the control plane and data plane. For instance, an SDN controller can update forwarding rules, modify actions, or retrieve statistics from a P4-enabled switch or router.

In our development environment, P4Runtime interfaces with BMv2 (Behavioral Model version 2) through the `simple_switch_grpc` variant, a specific extension of BMv2 that adds P4Runtime capabilities for control plane interaction. While the V1Model architecture defines the data plane structure that BMv2 implements, P4Runtime provides the control plane interface that manages this data plane at runtime.

The P4Runtime workflow starts with writing a P4 program that specifies packet headers, parsing logic, match-action tables, and processing rules. The program is compiled using a P4 compiler, such as p4c. This generates two key files: a P4.json file representing the data plane pipeline and a P4Info.txt file describing the control plane API. After compilation, these files are loaded into a P4-compatible switch, such as BMv2. While the switch understands the defined packet processing logic, specific forwarding rules are still absent.

P4Runtime allows an external controller to dynamically manage these rules by modifying match-action table entries and other runtime-configurable parameters, such as counters or meters. In a Mininet-based P4 project, automation scripts handle the compilation and loading of the JSON file into BMv2. P4Runtime issues control commands that insert, modify, or delete table entries, effectively managing packet processing in real time.

This three-part relationship (V1Model specification, BMv2 implementation, and P4Runtime control) creates a complete programmable networking environment. In this setup, the data plane structure is defined by P4 programs targeting V1Model, executed by BMv2, and dynamically controlled via P4Runtime. This separation of concerns

between data plane definition, implementation, and control is a fundamental principle of programmable networking. It enhances flexibility and maintainability, allowing dynamic updates to forwarding behavior, security policies, and traffic management without disrupting operations.

2.3.1 P4 Files

As seen in the previous sections, P4Runtime relies on two key files generated by the P4 compiler as part of the P4 program compilation process: P4.json and P4Info.txt. These files are generated from the P4 source code (.p4 files), ensuring consistency between the compiled program and the control plane's interpretation. These files describe the structure of the P4 program, including match-action tables, actions, headers, and metadata. These files are essential for the controller to interpret and interact with the compiled P4 pipeline, allowing it to modify match-action tables and actions dynamically. These files ensure consistency and validity in the interaction between the control plane and the data plane, enabling the controller to validate its commands.

2.3.2 P4.json

- This is a machine-readable JSON file that contains metadata about the P4 program.
- It defines all match-action tables, headers, actions, counters, meters, and other programmable entities that are part of the P4 program.
- P4Runtime uses this file to understand how to interact with the data plane and issue control commands.

2.3.3 P4Info.txt

- This is a human-readable text representation of the P4 program's control plane API in Protocol Buffer format.
- It contains information about tables, actions, and their parameters in a structured format.
- It is particularly useful for debugging or quickly referencing table structures, field names, and action parameters, such as in the case of diagnosing a mismatch between the control plane and data plane.
- In P4Runtime, the P4Info.txt file is used by controllers to understand the structure of the P4 program and validate control plane commands.

Both files are essential for P4Runtime controllers. The P4Info.txt file defines the control plane API, while the P4.json file provides implementation details. For instance,

a controller might load different P4Info.txt and P4.json file pairs for different types of switches (e.g., edge and core switches) if those switches run different P4 programs. This allows the controller to manage diverse switch types with different functionality while maintaining proper control over each P4 pipeline. Figure 10 and Figure 11 present examples of P4.json and P4Info.txt files respectively.

```
{
  "name": "MyIngress.set_nodeid",
  "id": 3,
  "runtime_data": [
    {
      "name": "nodeid",
      "bitwidth": 32
    }
  ],
  "primitives": [
    {
      "op": "assign",
      "parameters": [
        {
          "type": "field",
          "value": ["scalars", "metadata.node_id"]
        },
        {
          "type": "runtime_data",
          "value": 0
        }
      ],
      "source_info": {
        "filename": "pot-polka-core.p4",
        "line": 170,
        "column": 8,
        "source_fragment": "meta.node_id = nodeid"
      }
    }
  ]
}
```

Figure 10: P4.json Representation of the set_nodeid Action

```
actions {
  preamble {
    id: 17012265
    name: "MyIngress.set_nodeid"
    alias: "set_nodeid"
  }
  params {
    id: 1
    name: "nodeid"
    bitwidth: 32
  }
}
```

Figure 11: P4Info.txt Format Showing the `set_nodeid` Action

2.3.4 Setting Forwarding Rules

One of P4Runtime's core functions is inserting, modifying, or removing entries in match-action tables, which dictate packet processing based on criteria such as destination IP address, MAC address, or protocol type. The P4Runtime API provides methods like TableEntry modifications through the Write RPC, allowing controllers to programmatically update forwarding rules. This capability enables dynamic network reconfiguration in response to changing traffic patterns, security requirements, or application needs without interrupting network operations.

2.3.5 Monitoring Network Traffic

P4Runtime provides access to counters, meters, and registers that record packet statistics and maintain state, allowing network operators to analyze performance metrics and detect anomalies. The API supports reading these stateful elements through dedicated RPCs, enabling visibility into traffic patterns and device behavior. Additionally, P4Runtime enables real-time modifications to table entries without requiring a device restart, ensuring seamless network updates. While P4Runtime doesn't directly configure packet actions (since these are defined in the P4 program itself), it allows controllers to select which actions are applied by installing appropriate table entries, thereby enforcing network policies and optimizing traffic management.

2.4 PolKA Source Routing

Polynomial Key-based Architecture (PolKA) [6] is a novel source routing method that eliminates the need to maintain state information in network packets at each hop. Unlike traditional source routing techniques such as port switching, which require modifying the packet header at every hop, PolKA achieves fully stateless routing by leveraging polynomial arithmetic over the binary Galois Field GF(2) and principles of the residue number system (RNS). This approach reduces computational overhead and enhances scalability by ensuring that the route identifier remains unchanged throughout the packet's journey.
PolKA is based on three essential polynomial identifiers:

- routeID (R): Embedded in the packet and remains constant.
- nodeID (S): Assigned to each core switch.
- portID (O): Determines the output port for packet forwarding.

Together, these identifiers enable each node to make forwarding decisions independently, ensuring efficient routing. The routeID is embedded in the packet by the edge switch, and the forwarding operation in each core node is computed by taking the remainder of the routeID divided by the nodeID, which determines the appropriate portID for packet forwarding. This eliminates the need to rewrite the packet header at each hop, making packet forwarding significantly more efficient. The routing decision is based on:

$$o_i(t) = R(t) \bmod s_i(t) \tag{1}$$

A Galois Field (GF) is a mathematical structure used in fields such as error correction, cryptography, and polynomial arithmetic. It consists of a finite set of elements where operations like addition and multiplication are performed, ensuring the results stay within the field. The most common Galois Fields are GF(2), which contains the binary elements 0 and 1, and GF(p), where "p" is a prime number. These fields are closed under their operations, meaning the result of adding or multiplying two elements will also be an element of the same field. In PolKA Source Routing, GF(2) is used for stateless routing, leveraging polynomial arithmetic for efficient packet forwarding without requiring state maintenance at each hop.

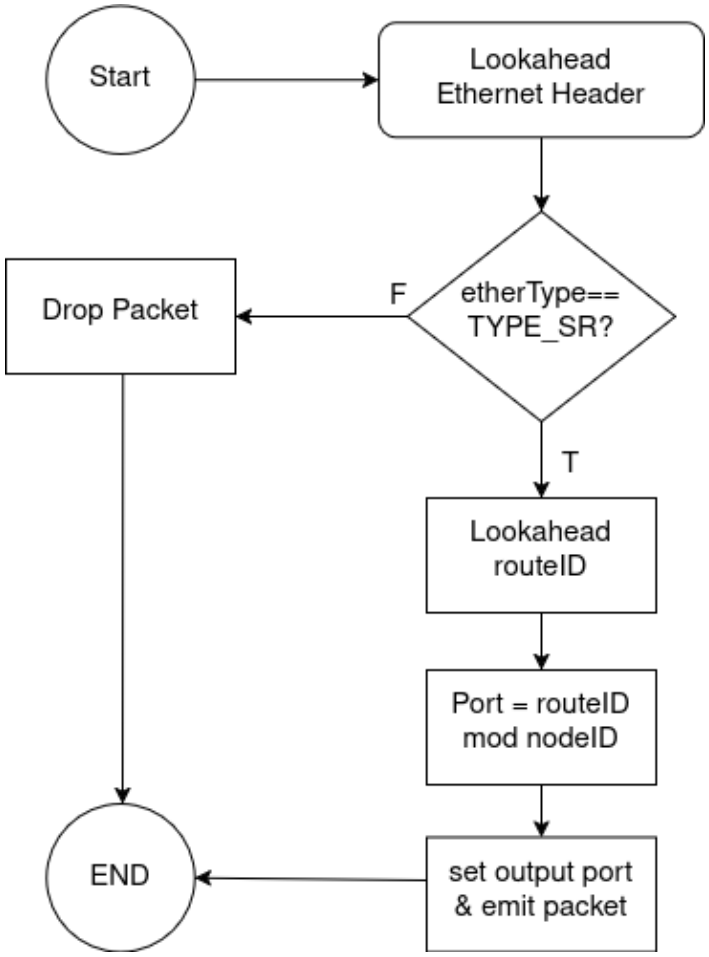


Figure 12: PolKA RNS-based SR

As shown in Figure 13, when the etherType is set to `TYPE_SR`, PolKA only requires read access to the packet headers. It uses the lookahead method in P4, which evaluates a set of bits from the input packet without advancing the packet index pointer. To determine the output port, the switch performs a modulo operation between the routeID in the packet and its own nodeID. Additionally, PolKA does not include information in the header to identify the last hop. However, this is not an issue in fabric networks, as the delivery of the packet to an edge switch marks the end of the source routing (SR) path. At the edge switch, the SR header is removed, and the etherType is updated to `TYPE_IPV4`.

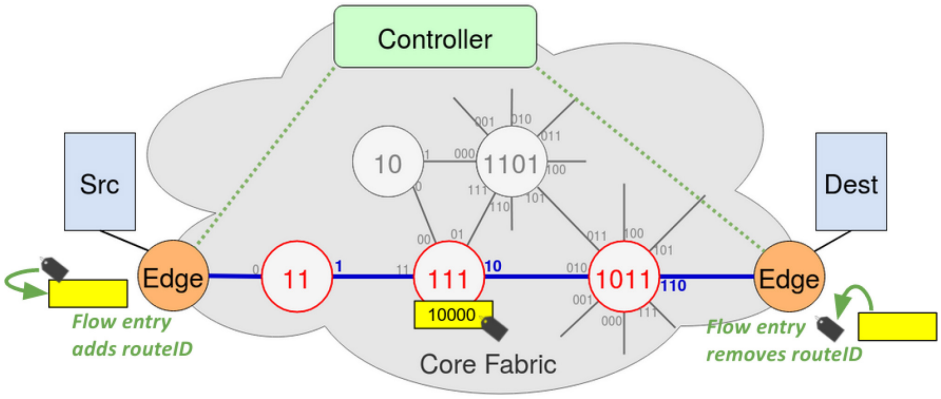


Figure 13: PolKA Source Routing

Figure 13 shows an example for a path composed of three core nodes, which received their nodeIDs from the Controller in a network configuration phase. For this example, the routeID, calculated according to the polynomial CRT, is R(t) = 10000

2.4.1 Calculating routeID

The routeID [7] in PolKA (Polynomial Key-based Architecture) is a unique identifier used for source routing in network fabrics. It remains unchanged throughout the entire path, allowing for stateless forwarding. Based on the previous definitions of nodeID and portID, the routeID is computed by the SDN controller using the Chinese Remainder Theorem (CRT), applied to polynomials over the Galois Field GF(2).

Step 1: Assigning Identifiers: Each node s_i in the network is assigned a unique nodeID, which is a pairwise co-prime polynomial over GF(2). The polynomials must satisfy:

$$\deg(s_i) \geq \lceil \log_2(\text{number of ports}) \rceil \tag{2}$$

where $\deg(s_i)$ is the degree of the polynomial assigned to node s_i. Each node also has a portID, which is a polynomial representing the output port.

Step 2: Compute the Polynomial Product: Define:

$$S = \{s_1(t), s_2(t), \ldots, s_N(t)\} \quad \rightarrow \text{the set of nodeIDs for the path.} \tag{3}$$

$$O = \{o_1(t), o_2(t), \ldots, o_N(t)\} \quad \rightarrow \text{the corresponding portIDs.} \tag{4}$$

The mod condition for the routeID is:

$$R(t) \equiv o_i(t) \pmod{s_i(t)}, \quad \text{for } i = 1, 2, \ldots, N \tag{5}$$

Using the Chinese Remainder Theorem (CRT), we get:

$$R(t) = \langle \tilde{R}(t) \rangle_{M(t)} \tag{6}$$

Where:

$$\tilde{R}(t) = \sum_{i=1}^{N} o_i(t) \cdot m_i(t) \cdot n_i(t) \quad \text{(CRT sum)} \tag{7}$$

$$M(t) = \prod_{i=1}^{N} s_i(t) \quad \text{(product of all node polynomials)} \tag{8}$$

$$m_i(t) = \frac{M(t)}{s_i(t)} \quad \text{(polynomial quotient)} \tag{9}$$

$$n_i(t) \cdot m_i(t) \equiv 1 \mod s_i(t) \quad \text{(via Extended Euclidean Algorithm)} \tag{10}$$

Step 3: Calculation: Consider a simple network with three core nodes S_1, S_2, S_3, with the following identifiers:

Node Identifiers (nodeID):

$$s_1(t) = t + 1 = (11)_2$$
$$s_2(t) = t^2 + t + 1 = (111)_2$$
$$s_3(t) = t^3 + t + 1 = (1011)_2$$

Port Identifiers (portID):

$$o_1(t) = 1 \quad \text{(binary: } (1)_2\text{)}$$
$$o_2(t) = t \quad \text{(binary: } (10)_2\text{)}$$
$$o_3(t) = t^2 + t \quad \text{(binary: } (110)_2\text{)}$$

Using CRT:

$$R(t) \equiv 1 \mod (t + 1)$$
$$R(t) \equiv t \mod (t^2 + t + 1)$$
$$R(t) \equiv (t^2 + t) \mod (t^3 + t + 1)$$

Solving these equations, the controller calculates:

$$R(t) = t^4 = (10000)_2$$

To enable practical deployment on commodity networking hardware, PolKA repurposes existing Cyclic Redundancy Check (CRC) hardware, which natively supports the polynomial modulo operations required for routing decisions. Since many programmable switches include CRC mechanisms, PolKA can be efficiently implemented in P4-based architectures.

2.4.2 CRC for Implementing mod Operation in P4

Commodity network hardware and the P4 language lack native support for polynomial or integer mod operations when operands are non-constant. To address this limitation, this work builds upon existing CRC mechanisms to develop a novel technique for executing polynomial mod operations directly in hardware. By leveraging the structure of CRC computations, this approach provides an efficient alternative that aligns with existing hardware capabilities, avoiding the need for explicit division operations.
In CRC calculations, the sender and receiver agree on a generator polynomial G, which is an $r+1$-bit pattern used for error detection. Given a data value D of d bits, the sender appends an additional r bits, R, to D to create a polynomial of $d+r$ bits. This polynomial is constructed so that it is divisible by G using modulo-2 arithmetic. The CRC code is obtained as the remainder when D, after being shifted left by r bits, is divided by G.
This approach proposes mapping the routeID to D and the nodeID to G. However, unlike conventional CRC encoding, PolKA does not shift the routeID. Since G has a degree of r, the challenge is addressed by decomposing the routeID into two parts:

$$\text{routeID} = D \cdot 2^r + \text{dif} \tag{11}$$

where dif represents the r least significant bits of the routeID.
The process begins by performing a right shift of the routeID by r bits, generating data D that serves as input for the CRC function. The bits displaced during the right shift operation (dif) are then reintegrated with the CRC remainder calculation to determine the final output port. The portID's degree remains below r, and through the polynomial division algorithm's uniqueness property, we confirm that the resulting polynomial matches the remainder from directly dividing the routeID by the nodeID. In the context of binary arithmetic, it is important to note that addition and subtraction operations yield results equivalent to the logical XOR operation.
The complete process unfolds in the following steps:

1. $G = \text{nodeID}, \quad r = \text{degree}(G)$
2. $D = \text{routeID} \div 2^r \quad$ (SHIFT RIGHT)
3. $\text{dif} = \text{routeID} - D \cdot 2^r \quad$ (SHIFT LEFT, XOR)
4. $R = \langle D \cdot 2^r \rangle_G \quad$ (CRC)
5. $\text{portID} = \text{dif} \oplus R \quad$ (XOR)

Thus, the switch calculates the output port using two shift operations, one CRC operation, and two XOR operations. This approach is computationally more efficient than performing division operations.

With this technique, PolKA can be deployed on P4 targets that allow the configuration of generator polynomials. Since P4 supports CRC operations through external libraries, the support for customized polynomials depends on architecture models and how specific targets implement them. The PSA and v1model architectures support customized polynomials of 16 and 32 bits. The software switch bmv2 and the hardware switch Tofino from Barefoot support customized CRC polynomials, while Netronome SmartNICs only support fixed CRC polynomials.

Routing Method	**State in Core**	**Forwarding Operation**	**Hardware Feasible**
Table-based Routing	Routing Table	Table Lookup	✓
Port Switching SR	Packet State	Header Rewrite (Pop)	✓
Integer RNS-based SR	None	Integer Mod	×
PolKA (Proposed)	None	Polynomial Mod (CRC)	✓

Table 1: Comparison of Different Routing Methods

Table 1 summarizes a comparison between routing methods, considering where the forwarding state is stored in the core network, what is the forwarding operation, and whether this operation can be implemented in commodity network hardware. Note that RNS-based SR methods are the only to offer fully stateless routing, because they do not update any state in the route label or in core nodes.

```
action srcRoute_nhop() {
    bit<16> nbase = 0;      // Base value for hash function
    bit<64> ncount = 4294967296*2;      // Range limit for hash output
    bit<16> nresult;        // Will store the CRC result
    bit<16> nport;      // Will store the final port value
    bit<160> routeid = (bit<160>) hdr.potPolka.routeId;     // Extract routeID

    bit<160> ndata = routeid >> 16;      // Calculate D = routeID / 2^r

    //Calculate dif = (routeID - D*2^r) using XOR
    bit<16> dif = (bit<16>) (routeid ^ (ndata << 16));

    hash(
      nresult,      // Will store the CRC result
      HashAlgorithm.crc16_custom,       // Using custom CRC16 algorithm
      nbase,        // Initial value (0)
      {ndata, meta.node_id},        // represents the polynomial being divided
      ncount        // parameter ensures the result is properly sized
    );

    nport = nresult ^ dif;      //Calculate portID = dif XOR R
    meta.port = nport[8:0];     //Extract routing information from the result

}
```

Figure 14: P4 Implementation of Polynomial Modulo Operation Using CRC16

The `srcRoute_nhop` action Figure 14, implements a hardware-efficient polynomial modulo operation using CRC16. This approach addresses the limitation in P4, where modulo operations with non-constant operands are not natively supported.

The algorithm operates in five steps:

1. Extract the higher-order bits of the routeID by right-shifting.
2. Capture the least significant bits by performing an XOR operation.
3. Perform a CRC16 hash calculation, using the nodeID as part of the input to simulate division by a polynomial.
4. Combine the CRC result with the previously captured bits using XOR.
5. Extract the final routing information (portID) from the resulting value.

This method enables consistent and deterministic routing decisions without requiring explicit division operations. It is particularly suitable for hardware implementations, while preserving the necessary mathematical properties for efficient path verification.

2.5 Shamir's Secret Sharing

Shamir's Secret Sharing (SSS) [8] is a cryptographic algorithm used to securely split a secret into multiple parts, called shares, which are distributed among a group of participants. The secret can only be reconstructed if a minimum number of participants, known as the threshold, come together and combine their shares. The algorithm ensures that no single participant has access to the entire secret, and the secret remains protected even if some shares are lost or compromised, as long as the threshold number of shares is still available.

Shamir's method is based on the principles of polynomial interpolation and modular arithmetic, where the secret is embedded into the coefficients of a polynomial, and the shares are points on that polynomial. Only a sufficient number of shares (defined by the threshold) can be used to reconstruct the original secret, ensuring both security and fault tolerance.

2.5.1 Mersenne Numbers

Mersenne numbers [9] are numbers of the form:

$$M_n = 2^n - 1 \tag{12}$$

where n is a positive integer. These numbers are named after the French mathematician Marin Mersenne.
For example, let's consider $n = 5$:

$$M_5 = 2^5 - 1 = 32 - 1 = 31 \tag{13}$$

- **Binary Representation:** $31 = (11111)_2$ (all ones in binary).
- **Prime Check:** 31 is a prime number, making M_5 a Mersenne prime.
- **Cryptography:** Large Mersenne primes are valuable in encryption algorithms such as RSA, which are fundamental for securing digital communications.
- **Efficient Computation:** Because Mersenne numbers are close to powers of two, they simplify modular arithmetic operations, making certain computations faster and more efficient.

2.5.2 Structure

Shamir's Secret Sharing algorithm operates through a series of steps designed to securely split and reconstruct a secret. First, a polynomial of degree $t-1$ is constructed, where t is the threshold number of shares required to reconstruct the secret. The constant term of the polynomial represents the secret itself, and the other coefficients are randomly chosen. This ensures that only the required number of shares are needed to reconstruct the secret. For example, if the secret is a number S, the polynomial could be written as:

$$f(x) = S + a_1 x + a_2 x^2 + \cdots + a_{t-1} x^{t-1} \tag{14}$$

where S is the secret and $a_1, a_2, \ldots, a_{t-1}$ are random coefficients. Next, the polynomial is evaluated at distinct points, typically using unique values as x-coordinates. Each evaluation point (x_i, y_i) forms a share, where x_i is the participant's identifier, and $y_i = f(x_i)$ is the corresponding share value. These shares are then distributed to the participants.

2.5.3 Reconstruction by Lagrange Polynomial Interpolation

To reconstruct the secret, participants must collaborate by pooling their shares. The minimum number of shares required to reconstruct the secret equals the threshold (t).

Using Lagrange interpolation, the secret can be reconstructed by combining the shares. Lagrange interpolation reconstructs $f(x)$ using t given shares $(x_1, y_1), (x_2, y_2), ..., (x_t, y_t)$. The polynomial is given by:

$$f(x) = \sum_{i=1}^{t} y_i \cdot \ell_i(x) \tag{15}$$

where the Lagrange basis polynomial is defined as:

$$\ell_i(x) = \prod_{j=1, j\neq i}^{t} \frac{x - x_j}{x_i - x_j} \tag{16}$$

where y_i is the share, and x_i is the corresponding x-coordinate of the share. This method ensures the secret can be reconstructed without revealing any information when fewer than t shares are available. The algorithm offers strong security guarantees. Any subset of fewer than t shares reveals no information about the secret, as it's impossible to deduce the polynomial or its constant term without sufficient shares. To find the secret, we evaluate this polynomial at $x = 0$:

$$S = f(0) = \sum_{i=1}^{t} y_i \cdot \ell_i(0) \tag{17}$$

where:

$$\ell_i(0) = \prod_{j=1\ j\neq i}^{t} \frac{-x_j}{x_i - x_j} \tag{18}$$

Computing the Lagrange basis polynomials for 4 nodes [10]:

$$\ell_0(x) = \frac{x - x_1}{x_0 - x_1} \cdot \frac{x - x_2}{x_0 - x_2} \cdot \frac{x - x_3}{x_0 - x_3} \tag{19}$$

$$\ell_1(x) = \frac{x - x_0}{x_1 - x_0} \cdot \frac{x - x_2}{x_1 - x_2} \cdot \frac{x - x_3}{x_1 - x_3}$$

$$\ell_2(x) = \frac{x - x_0}{x_2 - x_0} \cdot \frac{x - x_1}{x_2 - x_1} \cdot \frac{x - x_3}{x_2 - x_3}$$

$$\ell_3(x) = \frac{x - x_0}{x_3 - x_0} \cdot \frac{x - x_1}{x_3 - x_1} \cdot \frac{x - x_2}{x_3 - x_3}$$

Polynomial interpolation is a method used to construct a polynomial that passes through a given set of points. In the context of secret reconstruction, the polynomial function $f(x)$ is defined as a sum of terms, where each term consists of a known value y_i multiplied by a corresponding Lagrange basis polynomial $\ell_i(x)$. The general form of the polynomial interpolation equation is given by:

$$f(x) = \sum_{i=0}^{n-1} y_i \cdot \ell_i(x) = y_0\ell_0(x) + y_1\ell_1(x) + ... + y_{n-1}\ell_{n-1}(x) \tag{20}$$

```
action cal_secret(){
    meta.secret = (meta.y * meta.lpc);
}
apply {
    cal_secret();
    hdr.potPolka.secret = hdr.potPolka.secret + meta.secret;
}
```

Figure 15: P4 Implementation of Secret Reconstruction

Figure 15 illustrates a P4 implementation for secret reconstruction performed across four nodes. Each of the four nodes executes the `cal_secret` action, which calculates a partial secret by multiplying two metadata values, meta.y and meta.lpc, and storing the result in meta.secret. This value is then added to the `hdr.potPolka.secret` field. After the computation at each node, the updated secret is forwarded to the next hop, allowing the cumulative reconstruction of the final secret across the network path.

2.5.4 Secret Reconstruction Example

Let's consider an example where we require all 4 nodes to reconstruct the secret.

Step 1: Define the parameters: Let's assume:

- The secret $S = 20$
- A prime number $p = 53$ (a prime larger than the secret)
- We choose $k = 4$ (all 4 shares are required)

We generate a cubic polynomial with $a_0 = S$:

$$f(x) = a_0 + a_1x + a_2x^2 + a_3x^3 \bmod p \tag{21}$$

where:

$$a_0 = 20 \text{ (the secret)}$$
$$a_1 = 3 \text{ (coefficient)}$$
$$a_2 = 4 \text{ (coefficient)}$$
$$a_3 = 2 \text{ (coefficient)}$$

The polynomial becomes:

$$f(x) = 20 + 3x + 4x^2 + 2x^3 \bmod 53$$

Step 2: Generate shares: We evaluate $f(x)$ at 4 different values of x (never using $x = 0$).

Node x	**Share** (x, y)
1	$f(1) = 20 + 3(1) + 4(1) + 2(1) \bmod 53 = 29$
2	$f(2) = 20 + 3(2) + 4(4) + 2(8) \bmod 53 = 5$
3	$f(3) = 20 + 3(3) + 4(9) + 2(27) \bmod 53 = 13$
4	$f(4) = 20 + 3(4) + 4(16) + 2(64) \bmod 53 = 12$

So, the four shares are: $(1, 29), (2, 5), (3, 13), (4, 12)$

Step 3: Reconstruct the secret using Lagrange interpolation: To reconstruct S, we use Lagrange Interpolation. The Lagrange basis polynomial is:

$$l_i(x) = \prod_{j=1, j\neq i}^{t} \frac{x - x_j}{x_i - x_j} \bmod p \tag{22}$$

At $x = 0$, we compute:

$$S = \sum_{i=1}^{4} y_i \cdot l_i(0) \bmod p \tag{23}$$

Step 4: Compute each $l_i(0)$: For each share (x_i, y_i), the basis polynomial $l_i(0)$ is:

$$l_1(0) = \frac{(0-2)(0-3)(0-4)}{(1-2)(1-3)(1-4)} \bmod 53$$
$$l_2(0) = \frac{(0-1)(0-3)(0-4)}{(2-1)(2-3)(2-4)} \bmod 53$$
$$l_3(0) = \frac{(0-1)(0-2)(0-4)}{(3-1)(3-2)(3-4)} \bmod 53$$
$$l_4(0) = \frac{(0-1)(0-2)(0-3)}{(4-1)(4-2)(4-3)} \bmod 53$$

The final values are: $l_1(0) = 4$, $l_2(0) = 47$, $l_3(0) = 4$, $l_4(0) = 52$

Step 5: Compute the secret (S)

$$\begin{aligned} S &= (29 \times 4) + (5 \times 47) + (13 \times 4) + (12 \times 52) \bmod 53 \\ &= 116 + 235 + 52 + 624 \bmod 53 \\ &= 1027 \bmod 53 \\ &= 20 \end{aligned}$$

2.6 Compose the Onion

Onion composition in secret sharing is a cryptographic technique that enhances security by introducing a layered approach to both securing and revealing information. Unlike traditional secret sharing schemes, where a secret is divided directly into shares and reconstructed in a single step, onion secret sharing adds multiple layers of encryption or secrets. Each layer is dependent on the previous one, forming a structure that is reminiscent of the layers of an onion. A notable analogy to this structure can be found in the onion routing concept used in the Tor network, where each layer of encryption is peeled away by successive nodes to reveal the underlying data.
This technique is particularly useful in scenarios that require more granular control over access to sensitive information. For instance, in progressive secret reconstruction, secrets are revealed in a specific order, ensuring that not all information is accessible at once. Layered security models are also well-suited to onion secret sharing, where access control is enforced incrementally, meaning that each additional layer adds another level of security. Furthermore, hierarchical secret management can benefit from this model by ensuring that different secrets are protected at varying levels of access.

2.6.1 How Onion Composition Works in Secret Sharing

In traditional secret sharing schemes, such as Shamir's Secret Sharing, a secret is divided into multiple shares, and a threshold number of shares is required to reconstruct the secret. Onion composition, however, takes a step further by structuring multiple secrets in such a way that reconstructing one layer of secrets provides the information necessary to unlock the next. This layering mechanism ensures that a sequence of secrets must be reconstructed in order, with each new layer building upon the knowledge of the previous one. As a result, access to deeper secrets is progressively revealed.
At each stage, the outermost layer of the onion is the first to be accessible, while the inner layers require the prior layers to be unlocked before they can be accessed. This sequential unlocking process ensures that the secrets are only revealed in a controlled, step-by-step manner.

2.6.2 Mathematical Formulation of Onion Composition

Consider a set of secrets $S_1, S_2, \ldots, S_n$ that need to be shared within this onion-like structure. The first secret, S_1, is shared in the traditional manner using a secret sharing scheme. Subsequent secrets are encrypted using the previously shared secrets as part of a recursive process.
The first step in this onion composition is to define the outermost layer. A polynomial $f_1(x)$ is chosen such that the secret S_1 is its evaluation at $x = 0$. This polynomial will be used to distribute shares of S_1 among the participants. Once the outer layer

is shared, the next layer's secret is encrypted using the outer secret, forming an encapsulated structure that cannot be opened without knowing the prior layer.
Each layer of the onion can be represented by a polynomial $f_i(x)$, with $f_i(0) = E_{i-1}$, where E_{i-1} is the encrypted form of the secret from the previous layer. The recursive structure continues in this manner, where each layer's secret depends on the one before it.

2.6.3 Example of Onion Secret Sharing

Let's work through an example with three secrets, S_1, S_2, and S_3, in an onion-like structure.

Step 1: Define Secrets We begin by defining the three secrets:

$$S_1 = \text{First Layer Secret}$$
$$S_2 = \text{Second Layer Secret}$$
$$S_3 = \text{Final Layer Secret}$$

Step 2: Encrypt Inner Secrets Instead of directly sharing S_2 and S_3, we encrypt them using the prior secrets. This encryption ensures that each subsequent secret is hidden behind the previous one:

$$E_1 = \text{Encrypt}(S_2, S_1)$$
$$E_2 = \text{Encrypt}(S_3, S_2)$$

Here, S_2 is encrypted with S_1, and S_3 is encrypted with S_2.

Step 3: Create Onion Shares Next, we create the polynomials that will represent the shares of each layer. The first polynomial $f_1(x)$ is constructed such that its evaluation at $x = 0$ is S_1, and we distribute the corresponding shares. The second polynomial $f_2(x)$ is created such that $f_2(0) = E_1$, and shares are distributed for this polynomial. Similarly, $f_3(x)$ is created with $f_3(0) = E_2$, and shares for this layer are distributed to participants. Each of these polynomials corresponds to a layer of the onion structure.

Step 4: Reconstruction Process The reconstruction of secrets occurs step by step, with each layer being revealed in order:

1. First, participants reconstruct S_1 from the shares of $f_1(x)$.
2. Next, using S_1, they decrypt E_1 to obtain S_2.
3. Finally, with S_2, they decrypt E_2 to obtain S_3.

This process ensures that deeper layers cannot be accessed without unlocking the outer layers first.

The onion secret sharing scheme enforces a structured process of unlocking secrets, ensuring that the access to each deeper level of information is granted only after prior layers have been reconstructed and revealed. This mechanism can be particularly useful in environments where the sequence of information release must be controlled, such as hierarchical systems of trust or progressive access control in sensitive applications. By incorporating onion composition into secret sharing schemes, we can achieve not just secure distribution of information but also a higher degree of control over how and when sensitive data is exposed to participants.

Feature	**Onion Secret Sharing**	**Shamir's Secret Sharing**
Structure	Multi-layered (sequential layers)	Single-layer (direct reconstruction)
Reconstruction	Layer-by-layer sequential process	One-step combination of shares
Access Control	Hierarchical dependency	Flat threshold access
Security	Protected inner layers	Full exposure at threshold
Encryption	Required between layers	Not required
Share Size	Larger (encrypted layers)	Smaller (single polynomial part)
Computation	Higher (encryption + reconstruction)	Lower (polynomial interpolation)
Flexibility	Progressive disclosure	All-at-once disclosure
Use Case	Multi-stage secrets, hierarchical systems	Simple secret distribution
Example	Multi-layer voting system	Board member key sharing

Table 2: Comparison of Secret Sharing Methods

The table 2 outlines the key differences between Onion Secret Sharing and Shamir's Secret Sharing. While Shamir's method focuses on splitting a secret directly among participants with a flat threshold model, Onion Secret Sharing introduces multiple encrypted layers, enforcing a sequential reconstruction process. This layered approach provides finer-grained access control and stronger internal security, at the cost of higher computational and share size overhead. Each method serves different application needs depending on the required level of access complexity and security.

3 Deployment

Proof-of-Transit (PoT-PolKA) [11] is a secure and scalable packet forwarding verification system designed for programmable, path-aware networks. Its implementation was carried out methodically and validated through Mininet-based emulation alongside P4-programmable switches. The system setup, deployment strategy, experimental evaluation, and performance analysis collectively demonstrate that the proposed approach is practical and effective for real-world networking environments.

3.1 Seamless Path Migration and Dynamic Routing

One of PoT-PolKA's key advantages is its ability to facilitate seamless path migration without compromising packet verification. To evaluate this capability, the system was tested in a dynamic routing scenario where a traffic flow was diverted due to network congestion or link failure. The network was able to adapt to changing conditions by simply updating the routeID at the edge node, without requiring modifications at intermediate nodes. This approach preserves PoT-PolKA's security guarantees, minimizes convergence time, and ensures efficient traffic rerouting.
This dynamic path reconfiguration is particularly beneficial for software-defined networking (SDN) systems, where network policies and traffic engineering decisions can change rapidly. PoT-PolKA enables fast reallocation of network resources while maintaining compliance with predefined security and verification policies for every packet.

3.2 System Setup and Implementation

The deployment of PoT-PolKA involves edge nodes, core nodes, and an egress verification mechanism. A central aspect of the system is the integration of Proof-of-Transit (PoT) verification into a path-aware source routing framework. Unlike traditional PoT mechanisms, which typically rely on table-based routing, PoT-PolKA employs a Polynomial Key-based Architecture (PolKA) to encode the entire path within a unique route identifier (routeID). This approach eliminates the need for per-hop routing tables, thereby reducing overhead and improving scalability.
The implementation leverages the P4 programming language to define the packet forwarding behavior within the network. Each programmable switch is configured to interpret the routeID and apply modular arithmetic operations to determine the correct forwarding decision while simultaneously updating the PoT metadata at each hop. At the egress node, a verification process ensures that packets have traversed all required nodes in the correct sequence.
A significant innovation in the PoT-PolKA deployment is the use of Mersenne numbers to optimize modular arithmetic computations needed by Shamir's Secret Sharing Scheme (SSSS). Since P4 does not natively support integer modulo operations, leveraging

Mersenne primes enables efficient, hardware-compatible implementations that can operate effectively on existing programmable switch architectures.
The system design is structured around three major components: control plane computation, data plane configuration, and path verification at the egress node. To evaluate and validate the setup, Mininet is employed as a network emulation environment.

3.2.1 Mininet Environment

Mininet [12] is a powerful and widely used network emulator that enables the creation of virtual network environments on a single machine. It provides a lightweight, efficient, and scalable platform to model complex network topologies, allowing researchers, engineers, and students to develop, test, and experiment with network protocols, applications, and configurations without the need for physical infrastructure.
Mininet operates by leveraging Linux-based virtualization technologies such as network namespaces and virtual Ethernet (veth) pairs to create virtual hosts, switches, routers, and links. These virtual network elements behave similarly to real hardware, enabling realistic network behavior within an emulated environment. A key advantage of Mininet is its ability to support Software-Defined Networking (SDN) architectures, as it seamlessly integrates with OpenFlow-compatible controllers like POX, Ryu, OpenDaylight, and ONOS. This makes it an essential tool for testing SDN protocols, experimenting with custom network functions, and validating security mechanisms.
One of the defining features of Mininet is its flexibility in defining network topologies. Users can create predefined topologies such as single-switch, tree, or linear networks, or develop custom topologies using Python scripting. This programmability enables precise control over network configurations, including bandwidth constraints, link delays, and packet loss rates, making Mininet highly suitable for performance testing and network optimization studies.
The ability of Mininet to emulate large-scale networks with minimal computational resources makes it an attractive choice for academic research and industry applications. It provides an interactive command-line interface (CLI) that enables real-time network inspection, traffic generation, and debugging. Additionally, Mininet integrates well with external tools such as Wireshark for packet analysis and iperf for performance benchmarking.
Since it runs entirely on a single machine, Mininet offers a cost-effective alternative to deploying physical testbeds. It is widely adopted in educational environments for teaching networking concepts, as well as in research and development for validating new protocols and security mechanisms before deployment in production networks. Its adaptability, efficiency, and ease of use make it an indispensable tool for anyone working in networking, SDN, or network security domains.

```
{
    "hosts": {
        "h1": {
            "IP": "X.X.X.X/X",
            "MAC": "X:X:X:X:X:X"
        },
        "h2": {
            "IP": "X.X.X.X/X",
            "MAC": "X:X:X:X:X:X"
        }
    },
    "switches": {
        "s1": {},
        "s2": {},
        "s3": {},
        "s4": {}
    },
    "links": [
        ["h1", "s1-p1"],
        ["s1-p2", "s2-p1"],
        ["s2-p2", "s3-p1"],
        ["s3-p2", "s4-p1"],
        ["s4-p2", "h2"]
    ]
}
```

Figure 16: Custom Topology Configuration in JSON Format

The topology shown in Figure 16 illustrates a simple linear network composed of two hosts and four switches, connected sequentially through specified interfaces. This JSON structure defines the IP and MAC addresses for hosts, creates switch nodes, and sets up the link relationships needed to emulate the topology within Mininet.

3.2.2 Polynomial-based Path Verification in PoT-PolKA

A polynomial is associated with a collection of nodes that represent the path in Shamir's Secret Sharing Scheme (SSSS), which forms the foundation of the PoT-PolKA algorithm. A secret is chosen from a polynomial of degree n, with $n + 1$ points allocated to each node. These are called 'shares,' and they ensure that the secret can only be reconstructed by a specific set of nodes. The zero-degree coefficient of a private polynomial (Poly1), created at the edge nodes, represents the secret. A public polynomial (Poly2) is also defined, where the non-constant coefficients are kept private and only the constant coefficient (RND) is made public. Calculations involving Poly1, Poly2, Mersenne numbers, and secret coefficients are performed to guarantee secure path binding when a path is selected for verification. The use of Mersenne numbers in modular arithmetic ensures that the polynomial operations stay within a specific numerical range, providing both security and consistency across computations. There are specific conditions for choosing the polynomials: assuming that K_1 is the degree of poly_1 and K_2 is the degree of poly_2, with N nodes in the core, it is required that $K_1 < N$ and $K_2 < N$. Since polynomials of minimum degree 3 are used, the number

of core nodes must be at least 4. In Algorithm 1 and Algorithm 2, both $poly_1$ and $poly_2$ have a degree of 3, so the minimum number of core nodes required is 4.

3.2.3 Data Plane Configuration

The PoT-PolKA routeID acts as a key to verify the nodes along the defined path while updating PoT metadata at each hop. Instead of relying on traditional table-based routing, PoT-PolKA uses PolKA's residue number system (RNS) to encode paths. Core nodes forward packets based on the remainder of dividing the routeID by their nodeID, ensuring an efficient and stateless routing mechanism. Each core node updates a cumulative metadata label (CML) within the packet, which is then validated at the egress node. This approach eliminates the need for intermediate path storage while ensuring verification integrity.

3.2.4 Path Verification at the Egress Node

The final step in PoT-PolKA's design is path verification at the egress node. The verification process checks whether the computed cumulative metadata label (CML) matches the expected verification value. If there is any discrepancy, the packet is identified as a deviation attempt and discarded. This step ensures that traffic follows the authorized path without requiring excessive state storage or complex table lookups, thus improving both security and scalability.

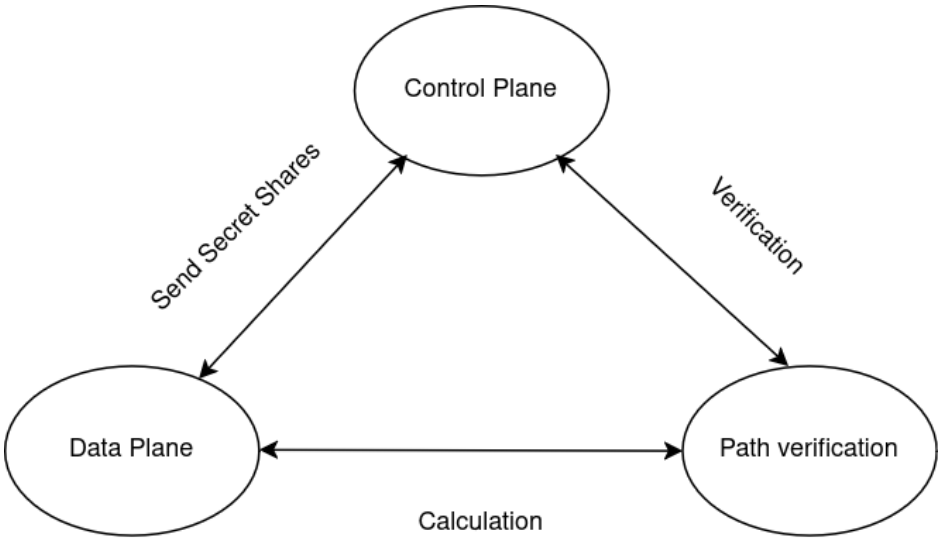


Figure 17: PoT Diagram

3.3 Linear Topology

In a network configuration known as a linear topology, nodes (or devices, such as switches or routers) are arranged in a straight line, sequentially. This topology typically consists of a series of interconnected nodes, each connected to two other nodes, with the exception of the two end nodes, which are each connected to only one node. Each node in a linear topology is responsible for forwarding data to the next node along the path, and data flows through the nodes in a predetermined order. In this setup, two edge nodes serve as ingress and egress points, while four core nodes act as intermediate nodes. Initially, P4Runtime (as described in Chapter 2, Section 2.3) is used as the controller to populate the tables and create the shares of the secret. Then, the P4 programming language (as discussed in Chapter 2, Section 2.2) is employed to work with the parameters derived from the controller to calculate the CML and portID.

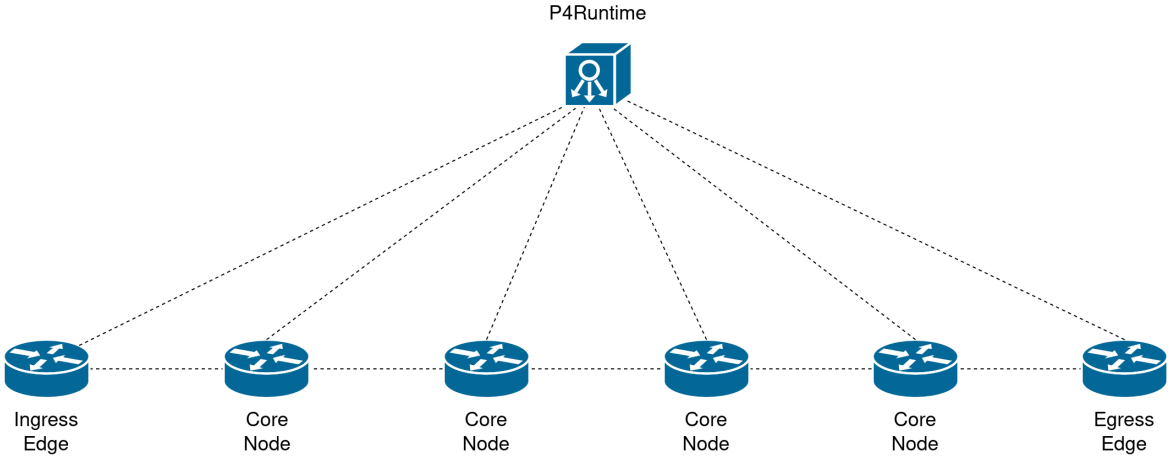


Figure 18: Linear Topology

P4Runtime: SECRET shares are generated and used to populate the tables.

Ingress Edge: Embeds the PoT-PolKA header into the packet.

Core Node: Calculates the CML and port ID.

Egress Edge: Reconstructs the SECRET to perform the verification phase.

3.4 P4Runtime Deployment

After describing the structure of P4Runtime in Chapter 2, Section 2.3, this section details my implementation of P4Runtime for deploying the PoT-PolKA mechanism across a network of programmable switches.

3.4.1 P4Runtime Architecture and Setup

P4Runtime follows a client-server architecture where the controller acts as a client that communicates with P4-programmable switches (servers) via gRPC. The controller uses the P4Info file (generated during P4 program compilation) to understand the structure of tables, actions, and other programmable elements defined in the P4 program.
The deployment begins with loading the P4Info and JSON files that define the data plane behavior for both edge and core switches:

```
# Set up P4Runtime client for core switches
p4info_helper_core = p4runtime_lib.helper.P4InfoHelper(p4info_core)

# Set up P4Runtime client for edge switches
p4info_helper_edge = p4runtime_lib.helper.P4InfoHelper(p4info_edge)
```

Figure 19: P4Runtime Initialization Code

Figure 19 shows that the P4Info helper objects provide methods to interact with P4 entities (tables, actions, etc.) using their names rather than numeric IDs, simplifying the controller implementation.

3.4.2 Switch Connection Management

To establish communication with the switches, we create P4Runtime connections for each switch in our topology. First, we define a configuration dictionary that contains the connection details for each switch:

```
SWITCH_CONFIG = {
    'e1': {'name': 'e1', 'address': '127.0.0.1:50051', 'device_id': 0},
    's1': {'name': 's1', 'address': '127.0.0.1:50053', 'device_id': 2},
    's2': {'name': 's2', 'address': '127.0.0.1:50054', 'device_id': 3},
    's3': {'name': 's3', 'address': '127.0.0.1:50055', 'device_id': 4},
    's4': {'name': 's4', 'address': '127.0.0.1:50056', 'device_id': 5},
    'e2': {'name': 'e2', 'address': '127.0.0.1:50052', 'device_id': 1},
}
```

Figure 20: Switch Configuration Dictionary

```
s1 = p4runtime_lib.bmv2.Bmv2SwitchConnection(
    name=SWITCH_CONFIG['s1']['name'],
    address=SWITCH_CONFIG['s1']['address'],
    device_id=SWITCH_CONFIG['s1']['device_id'],
    proto_dump_file='logs/s1-p4runtime-requests.txt')
s1.log_callback = lambda msg: log_to_console_and_file('s1', msg)

# Similar connections for other switches...

# Send master arbitration update message for all switches
for switch in [e1, e2, s1, s2, s3, s4]:
    switch.MasterArbitrationUpdate()
    time.sleep(0.5)
```

Figure 21: Switch Connection Establishment

Figure 21 shows that, after using this configuration, we establish connections to both edge and core switches. The `MasterArbitrationUpdate` message is essential in P4Runtime, as it establishes which controller has write access to the switch. In our implementation, we ensure that our controller is the master for all switches. The configuration demonstrates that we are connecting to six switches: two edge switches (e1, e2) and four core switches (s1, s2, s3, s4), each with a unique device ID and running on localhost with different port numbers.

Figure 22 illustrates the communication sequence of the P4Runtime protocol between a controller and a BMv2 software switch. The protocol consists of five key steps: (1) establishing a gRPC connection, (2) performing master arbitration, (3) setting the pipeline configuration, (4) writing table entries to the switch, and (5) reading table entries from the switch, followed by the switch's response back to the controller.

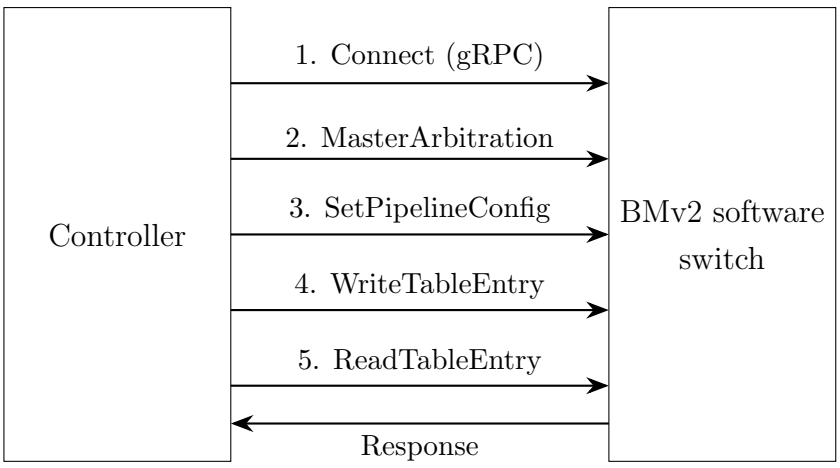


Figure 22: P4Runtime Protocol Flow between Controller and Switch

3.4.3 Installing P4 Programs

After establishing connections, we install the P4 programs on the switches using the `SetForwardingPipelineConfig` RPC:

```
# Install the P4 program on the edge switches
e1.SetForwardingPipelineConfig(p4info=p4info_helper_edge.p4info,
                               bmv2_json_file_path=bmv2_file_path_edge)
print("Installed P4 Program using SetForwardingPipelineConfig on e1")

e2.SetForwardingPipelineConfig(p4info=p4info_helper_edge.p4info,
                               bmv2_json_file_path=bmv2_file_path_edge)
print("Installed P4 Program using SetForwardingPipelineConfig on e2")

# Install the P4 program on the core switches
for switch in [s1, s2, s3, s4]:
    switch.SetForwardingPipelineConfig(p4info=p4info_helper_core.p4info,
                                       bmv2_json_file_path=bmv2_file_path_core)
    switch.p4info = p4info_helper_core.p4info
    print(f"Installed P4 Program using SetForwardingPipelineConfig on {switch.name}")
```

Figure 23: Installing P4 Programs on Switches

Figure 23 shows the operation to configure the switches with the appropriate P4 program, enabling them to process packets according to our defined behavior. Note that we use different P4 programs for edge and core switches, as they perform different functions in our PoT-PolKA implementation.

3.4.4 Table Entry Management

After installing the P4 programs, the next step is to populate the match-action tables to define the packet processing behavior. P4Runtime enables dynamic management of these tables by allowing us to insert, modify, and delete table entries at runtime. This flexibility ensures that packet forwarding and processing behavior can be adjusted on the fly, making the network more adaptable to changing conditions.

3.4.5 IPv4 Forwarding Rule

```
def add_ipv4_forward_rule(p4info_helper_edge, switch, dst_ip_addr,
                          dst_mac_addr, egress_port):
    table_entry = p4info_helper_edge.buildTableEntry(
        table_name="MyIngress.ipv4_lpm",
        match_fields={
            "hdr.ipv4.dstAddr": (dst_ip_addr, 32)
        },
        action_name="MyIngress.ipv4_forward",
        action_params={
            "dstAddr": dst_mac_addr,
            "port": egress_port
        }
    )
    switch.WriteTableEntry(table_entry)
    print(f"Installed IPv4 forwarding rule on {switch.name}")
```

Figure 24: Basic IPv4 Forwarding Table Configuration

Figure 24 shows a function that configures IPv4 forwarding rules in P4 switches. The function creates a table entry that matches destination IP addresses with a 32-bit prefix (exact match) and associates them with specific forwarding actions. When a packet's destination IP matches the rule, the switch forwards it to the specified MAC address through the designated egress port.

3.4.6 Source Routing Forwarding in P4Runtime

Source routing in P4 refers to a packet forwarding method where routing information is embedded directly in packet headers by the source node. When implemented in a P4-programmable device, the P4 program defines custom headers to carry this routing information, along with the logic to parse these headers and make forwarding decisions based on their content. P4Runtime, as the control plane API, is used to install and manage the tables and other control plane state that support the source routing implementation. While the actual source routing logic is defined in the P4 program and executed in the data plane, P4Runtime enables controllers to configure the necessary match-action tables, registers, or other stateful elements that might be needed. This approach allows for explicit path control without requiring routing table lookups at each hop, which can be beneficial for traffic engineering, policy enforcement, or providing service guarantees. In P4-based networks, source routing leverages the protocol-independence of P4 to implement custom header formats and processing logic tailored to specific network requirements.

```python
def Edge_Table_forward(p4info_helper_edge, switch, dst_ip_addr, rnd, cml,
                       dst_mac_addr, routeId, egress_port, secret):
    table_entry = p4info_helper_edge.buildTableEntry(
        table_name="MyIngress.process_tunnel_encap.tunnel_encap_process_sr",
        match_fields={
            "hdr.ipv4.dstAddr": (dst_ip_addr, 32)
        },
        action_name="MyIngress.process_tunnel_encap.add_sourcerouting_header",
        action_params={
            "port": egress_port,
            "dmac": dst_mac_addr,
            "routeIdPacket": routeId,
            "rnd": rnd,
            "cml": cml,
            "secret": secret
        }
    )
    switch.WriteTableEntry(table_entry)
    print("Installed ingress tunnel rule on %s" % switch.name)
```

Figure 25: Edge Switch Source Routing Configuration

Figure 25 shows the P4Runtime implementation for source routing forwarding. The function `Edge_Table_forward()` demonstrates how to programmatically configure the source routing table. It builds a table entry that matches packets with a specific destination IP address and applies the `add_sourcerouting_header` action. This action encapsulates the original packet with a source routing header containing parameters like route ID, random number (rnd), cumulative value (cml), and a secret value, which is initially set to 0. The control plane code uses the P4Runtime API to insert the entry into the edge switch's forwarding table, enabling the dynamic configuration of secure source routing paths in the network.

3.4.7 Core Switch Configuration

Each core switch needs to be configured with the polynomial parameters for the PoT-PolKA algorithm:

```
def Core_Table_Forward(p4info_helper_core, switch, x_forward, y_forward,
                       lpc_forward, Poly2_forward, mersenne_core,
                       mersenne_b_core, pot_k_forward):
    table_entry = p4info_helper_core.buildTableEntry(
        table_name="MyIngress.pot_param",
        match_fields={
            "meta.pot_k": pot_k_forward
        },
        action_name="MyIngress.initialize_param",
        action_params={
            "x": x_forward,
            "y": y_forward,
            "lpc": lpc_forward,
            "mersenne": mersenne_core,
            "mersenne_b": mersenne_b_core,
            "poly2": Poly2_forward
        }
    )
    switch.WriteTableEntry(table_entry)
    print("Assigned secret parameters on %s" % switch.name)
```

Figure 26: PoT-PolKA Parameter Configuration

Figure 26 illustrates the configuration of cryptographic parameters for the Proof of Transit (PoT) mechanism in the PolKA architecture. This function configures core switches with the necessary parameters to perform in-network verification operations. The table entry matches a key value and initializes multiple cryptographic parameters used in the verification process, enabling secure path validation as packets traverse the network.

3.4.8 NodeID Distribution

Each core node dynamically obtains its nodeID, which (as described in Chapter 2, Section 1.4) is then used to calculate the portID, allowing for the distinction of the output port.

```
def configure_node_id(p4info_helper_core, switch, node_id, routeId):
    print("Assign NodeID on Switch %s" % switch.name)
    table_entry_forward = p4info_helper_core.buildTableEntry(
        table_name="MyIngress.node_config",
        match_fields={
            "hdr.potPolka.routeId": routeId
        },
        action_name="MyIngress.set_nodeid",
        action_params={
            "nodeid": node_id,
        }
    )
    switch.WriteTableEntry(table_entry_forward)
    print(f"Node ID set to: {node_id}")
```

Figure 27: Node ID Configuration for Core Switches

Figure 27 demonstrates the P4Runtime API function that assigns unique node identifiers to core switches in the network. The function configures a match-action table entry, where packets with a specific route identifier (routeId) trigger the assignment of a predefined node ID. This configuration is essential for the PoT-PolKA mechanism, as each switch needs a distinct identity to participate correctly in the path verification process.

```
Found actions: name=MyIngress.set_nodeid, alias=set_nodeid, id=17012265
Encoding value: 65579
Type of value: <class 'int'>
Bitwidth: 32
Expected byte length: 4
Encoded length: 4
Encoded bytes: 0001002b
Installed node_id on s1
Node ID set to: 65579
```

Figure 28: P4Runtime Controller Output When Setting Node ID

Figure 28 shows the P4Runtime controller setting a unique identifier (65579) on switch s1 using the `MyIngress.set_nodeid` action. The controller encodes the 32-bit value and confirms the successful installation of the node ID parameter.

Algorithm 1: Configure nodeID in P4Runtime

Data: p4info_helper_core, switch, nodeID, routeId

Result: Configure the switch with the given nodeID

a. Table name: "node_config";

b. Match field: "routeId";

c. Action name: "set_nodeID";

d. Action parameter: "nodeID";

Algorithm 2: Configure nodeID in Data Plane

Data: routeId (table key), nodeID

Result: Assign the nodeID in the data plane

1. Define table `node_config`;
   a. Key: routeId (exact match);
   b. Action name: `set_nodeID`;
   c. Size: 2 entries;
   d. Default action: `set_nodeID(0)`;
2. Define action `set_nodeID(nodeID)`;
   a. Assign `meta.node_id` $\leftarrow$ nodeID;
   b. Log message: "Switch nodeID set to `meta.node_id`";
3. End;

Algorithm 1 outlines the control plane procedure, where the P4Runtime API is used to construct and insert table entries into the switch. This process involves specifying the exact table structure, match fields, and action parameters that will be used for packet processing. The controller uses this method to dynamically program switches based on network requirements.

Algorithm 2 details the corresponding data plane implementation in P4 language. It shows how the table is defined within the switch's processing pipeline, including its match key structure, available actions, and default behavior. When packets arrive with matching routeId values, the switch assigns the corresponding nodeID to a metadata field, which is then used in subsequent processing stages. The default action with nodeID=0 ensures that packets with unrecognized routeIds still receive basic processing. This two-part approach—control plane configuration and data plane implementation—demonstrates the separation of concerns in programmable networks, where the controller determines the policy (which nodeID to assign) while the data plane efficiently executes the packet processing according to that policy.

3.4.9 Reading Table Entries and Monitoring

P4Runtime allows us to read the current state of tables to verify our configuration and for debugging purposes. We implemented a function to read and display all table entries from a switch:

```python
def readTableRules(p4info_helper, switch):
    folder_name = 'table_entries'
    if not os.path.exists(folder_name):
        os.makedirs(folder_name)
    filename = os.path.join(folder_name, f"{switch.name}_table_rules.txt")
    with open(filename, 'w') as f:
        print(f'\n----- Reading table rules for {switch.name} -----')
        try:
            for response in switch.ReadTableEntries():
                for entity in response.entities:
                    entry = entity.table_entry
                    table_name = p4info_helper.get_tables_name(entry.table_id)
                    print(f'{table_name}: ', end=' ')

                    # Process and display match fields
                    for m in entry.match:
                        match_field_name = p4info_helper.get_match_field_name(
                            table_name, m.field_id)
                        match_field_value = format_value(
                            p4info_helper.get_match_field_value(m))
                        print(f'{match_field_name}={match_field_value}', end=' ')

                    # Process and display action and parameters
                    action = entry.action.action
                    action_name = p4info_helper.get_actions_name(action.action_id)
                    print(f'-> {action_name}', end=' ')

                    for p in action.params:
                        param_name = p4info_helper.get_action_param_name(
                            action_name, p.param_id)
                        param_value = format_value(p.value)
                        print(f'{param_name}={param_value}', end=' ')

                    print()
                    # Also write to file
                    # ... [file writing code omitted for brevity]
        except grpc.RpcError as e:
            print(f"gRPC Error reading rules from {switch.name}: {e}")
```

Figure 29: Reading Table Entries from Switches

Figure 29 displays how to read P4 table entries from network switches. It extracts match fields, actions, and parameters from each entry, enabling the user to monitor the current state of the switch tables. The `readTableRules()` function operates on the control plane and interacts with the switch via P4Runtime's gRPC interface. As such, it runs **out-of-band** from the data plane and does **not interfere with packet forwarding performance**. This separation of concerns ensures that operational diagnostics and state monitoring can be performed without impacting the high-throughput data processing on the switches.

3.4.10 Error Handling and Connection Management

Proper error handling and connection management are crucial for a robust P4Runtime implementation. Our controller includes functions to handle gRPC errors and to properly shut down switch connections:

```
def printGrpcError(e):
    print("gRPC Error:", e.details(), end="")
    status_code = e.code()
    print(" (%s)" % status_code.name, end="")
    traceback = sys.exc_info()[2]
    print(" [%s:%d]" % (traceback.tb_frame.f_code.co_filename, traceback.tb_lineno))

# In the main function's finally block:
finally:
    # Shutdown switch connections at the end
    ShutdownAllSwitchConnections()
    print("All switch connections shut down successfully.")
```

Figure 30: Error Handling and Connection Management

Proper connection shutdown, as shown in Figure 30, is essential to release system resources such as memory, sockets, and file descriptors, preventing resource leaks that could otherwise degrade system performance over time. In a P4Runtime environment, where the controller may frequently establish and terminate multiple gRPC connections to switches, improper shutdown can lead to lingering connections, port conflicts, or unexpected behavior during controller restarts. By systematically closing all switch connections at the end of execution, we ensure that the controller can be cleanly restarted without errors, maintain stable communication with the network elements, and avoid potential disruptions in network management operations.
The `printGrpcError` function is essential for capturing gRPC errors that might occur during communication with switches. By logging detailed information, including the error message, status code, and traceback, this function enables the controller to identify and handle issues effectively. This contributes to a more reliable and resilient controller, capable of recovering from transient network failures or misconfigurations.
Effective error handling and connection management not only enhance the reliability of the controller but also ensure that PoT-PolKA can maintain its high-performance standards. By avoiding resource leaks and minimizing connection disruptions, the controller can sustain optimal communication with the network, which is crucial for maintaining path verification integrity and ensuring smooth traffic rerouting during network failures or topology changes.

3.4.11 Summary of P4Runtime Deployment

Our P4Runtime implementation successfully deploys the PoT-PolKA mechanism across a network of programmable switches. The controller:

- Establishes connections to all switches in the topology.
- Installs the appropriate P4 programs on edge and core switches.
- Configures the necessary tables for basic forwarding and PoT-PolKA functionality.
- Sets up the polynomial parameters required for path verification.
- Provides mechanisms to verify the configuration through table entry reading.
- Handles errors and properly manages switch connections.

This implementation demonstrates how P4Runtime enables the dynamic deployment and configuration of complex network functions like path verification without requiring changes to the underlying hardware.

3.5 Define Parameters for Secret Sharing

We aim to create our secret shares using a polynomial. For an introduction to deploying Shamir's Secret Sharing Scheme, refer to Chapter 2, Section 2.5.

3.5.1 Poly1: Private Polynomial for Secret Sharing

Poly1 is a private polynomial used in the PoT-PolKA mechanism to distribute secret values among nodes for path verification. It is constructed using Shamir's Secret Sharing Scheme (SSSS) and serves as a fundamental component in ensuring the authenticity of packet transit. The polynomial is generated at the control plane and follows a structured process to guarantee both security and correctness.
Poly1 is a polynomial of degree n, where (n) is at least one less than the number of nodes in the path. It is represented as:

$$\text{Poly1}(x) = a_n x^n + a_{n-1} x^{n-1} + \ldots + a_1 x + S \quad \text{where } n = t - 1 \tag{24}$$

where:

- $a_n, a_{n-1}, \ldots, a_1$ are randomly chosen coefficients that remain private.
- S is the zero-degree coefficient (constant term) and represents the secret key used for final verification at the egress node.

Poly1 is constructed such that the coefficients $a_n, a_{n-1}, \ldots, a_1$ remain private and can only be used by nodes who receive their corresponding shares. The constant term S serves as the secret key used to verify packet authenticity at the egress node.

Algorithm 3: Generate Polynomial1 Points

Data: mersenne, x_values
Result: x_list - y_list
Set coefficient ← Value;
Set secret ← Value;
Initialize x_list ← [ ] ; `// Empty list for x values`
Initialize y_list ← [ ] ; `// Empty list for y values`
for *each x in x_values* **do**
  $P(x) \leftarrow S + \sum_{i=1}^{n} (a_i x^i)$;
  $y \leftarrow P(x) \mod$ mersenne;
  Append x to x_list;
  Append y to y_list;
end
return *x_list, y_list*

Algorithm 3 describes how to generate a set of points based on a secret polynomial (Poly1) for Shamir's Secret Sharing. The process involves evaluating a simple polynomial at each x value using predefined coefficients and a secret value. Each computed y value is reduced modulo a Mersenne prime to ensure boundedness and cryptographic properties. The output is two lists: the original x inputs and their corresponding y values, which can then be distributed among network nodes for secret reconstruction. The control plane generates $n+1$ unique points on the polynomial curve as follows:

$$(x_0, \text{Poly1}(x_0)),\ (x_1, \text{Poly1}(x_1)),\ \ldots,\ (x_n, \text{Poly1}(x_n)) \tag{25}$$

We define $x_0 = 1$, $x_1 = 3$, $x_2 = 5$, and $x_3 = 7$. The polynomial evaluations are:

$$\begin{aligned}
\text{Poly1}(1) &= 19 = (x_0, y_0) = (1, 19) \\
\text{Poly1}(3) &= 3 = (x_1, y_1) = (3, 3) \\
\text{Poly1}(5) &= 10 = (x_2, y_2) = (5, 10) \\
\text{Poly1}(7) &= 29 = (x_3, y_3) = (7, 29)
\end{aligned}$$

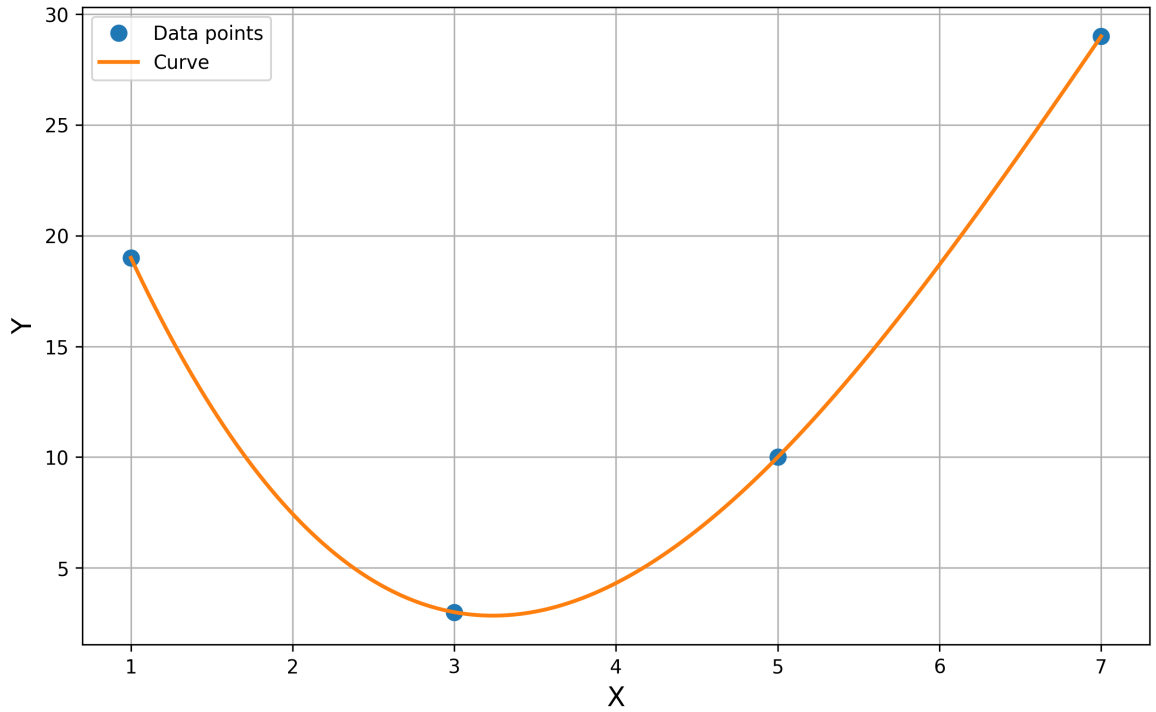


Figure 31: Polynomial Curve for Path Verification

Figure 31 illustrates the polynomial curve, which represents the secret shared across the network. Each core switch in the network is assigned a unique point on this curve. As packets traverse the network, each switch verifies the authenticity of the packet by ensuring it corresponds to its assigned point, contributing to the path verification process.

3.5.2 Poly2: Public Polynomial for Metadata Updates

Poly2 is a public polynomial utilized in PoT-PolKA to dynamically update the Cumulative Metadata Label (CML) at each hop along the network path. CML is embedded into the packet at the ingress edge and initially assigned a value of 0. Unlike Poly1, which distributes secret shares for cryptographic validation, Poly2 enables secure metadata modification. The constant coefficient of Poly2 is public, while the remaining coefficients are kept private to prevent unauthorized tampering.
Poly2 is defined as:

$$\text{Poly2}(x) = b_n x^n + b_{n-1} x^{n-1} + \ldots + b_1 x + \text{RND} \quad \text{where } n = t - 1 \tag{26}$$

Here:

- $b_1, \ldots, b_n$ are randomly chosen and secret.
- `RND` is a public constant generated at the ingress node to ensure uniqueness across flows.

Algorithm 4: Generate Polynomial2 Values

Data: `x_values`, `mersenne`, `RND`
Result: `poly2_list`
Initialize `poly2_list` $\leftarrow [\,]$; `// Empty list for Poly2 values`
for *each* x *in* ***`x_values`*** **do**
  Compute: $P(x) \leftarrow (b_3 x^3 + b_2 x^2 + b_1 x + \texttt{RND})$;
  Apply modulus: $P(x) \leftarrow P(x) \bmod \texttt{mersenne}$;
  Append $P(x)$ to `poly2_list`;
end
return *`poly2_list`*

3.5.3 CML Calculation

Each core node i along the path receives a corresponding pair $(x_i, \text{Poly2}(x_i))$ [11], similar to Poly1. Like Poly1, LPCs are also incorporated. These values are used in metadata updates, ensuring that the packet accumulates a verifiable transit signature. The update formula at each hop i is:

$$\text{CML} = \Big(\text{CML} + \big(\text{Poly1}(x) + \text{Poly2}(x)\big) \times \text{LPC}\Big) \bmod M \tag{27}$$

where:

- CML is the Cumulative Metadata Label, continuously updated along the path.
- Poly1(x) and Poly2(x) represent the polynomial values at the current core node.
- LPC is the Lagrange Polynomial Constant(as described in Chapter 2, Section 2.5.3), which is computed by the Controller and communicated to the nodes.
- M is Mersenne prime number, ensuring efficient modular arithmetic.

This approach enables secure and verifiable metadata updates without exposing sensitive polynomial coefficients, ensuring both functionality and security in network operations.

```
action calc_cml () {
    meta.new_cml = (meta.y + meta.poly2) * meta.lpc;
    meta.new_cml = (meta.new_cml & meta.mersenne) + (meta.new_cml >> meta.mersenne_b);
    if (meta.new_cml > meta.mersenne) {
        meta.new_cml = meta.new_cml - meta.mersenne;
    }
    meta.new_cml = hdr.potPolka.cml + meta.new_cml;
}

apply {
    calc_cml();
}
```

Figure 32: P4 Action Definition for CML

The P4 code in Figure 32 implements the CML calculation in programmable data planes. The action `calc_cml()` performs the following operations:

1. It calculates `(meta.y + meta.poly2) * meta.lpc`, where `meta.y` is Poly1(x) and `meta.poly2` is Poly2(x).
2. It performs an efficient modulo using Mersenne prime properties via bitwise operations.
3. If the result exceeds the Mersenne prime, it subtracts it to remain in the valid range.

4. It adds the result to the existing CML value in `hdr.potPolka.cml`, enforcing cumulative behavior.

To enable a feasible implementation of SSSS using the P4 language, there is a technique to calculate the modulo operation with Mersenne numbers [9], which follow the pattern $(2^B) - 1$. A number modulo a Mersenne number can be calculated by a shift bitwise operation (<<). Algorithm 5 shows how the calculation of $k \bmod p$ can be executed with elementary operations, where p is a Mersenne number, k is smaller than $(1 << (2 * B)) - 1$, and B is the power of two of the Mersenne number $((2^B) - 1)$. To clarify, assuming that $p = 31$ and $B = 5$, it means that k should be smaller than $(1 << 10) - 1 = 1023_{10} = 1111111111_2$.

Algorithm 5: Modulo Mersenne Number

Data: k - Input integer
p - Integer value
B - Integer value
Result: i - Integer value

1. $i \leftarrow (k \& p) + (k \gg B)$;
2. **if** $i \geq p$ **then**
   a. $i \leftarrow i - p$;
3. **return** i;

4 Proof of Concept

In this section, we demonstrate the practical feasibility of the proposed Proof of Transit (PoT) mechanism by implementing it in a controlled environment. We validate that packets can be securely tracked along their intended network paths, and we show that any deviation from the expected path can be detected and handled in real time through our verification process.

4.1 Verification Phase

At the verifier node, verification is performed by checking whether the CML in the packet header matches the computed VERIFY value. The RND remains fixed during the path, but the CML is computed hop by hop. So, in the egress edge, the verifier applies Equation (28). Given that the VERIFY computed in the edge is equal to CML in the packet header, the path verification is confirmed.

$$\text{VERIFY} = (S + \text{RND}) \bmod M \tag{28}$$

where:

- S is the secret.
- RND is the random number stored in the packet header.
- M is a Mersenne prime number, ensuring computational efficiency.

If the computed VERIFY matches the final CML in the packet header, the packet is validated as having followed the correct path. Otherwise, it is identified as invalid and dropped. This ensures that unauthorized path deviations are detected and mitigated in real time.

P4Runtime Controller Setup

```
Encoding value: 31 - (Mersenne Prime Number)
Type of value: <class 'int'>
Bitwidth: 16
Expected byte length: 2
Encoding value: 50 - (RND)
Type of value: <class 'int'>
Bitwidth: 16
Expected byte length: 2
Encoding value: 0 - (CML)
Type of value: <class 'int'>
Bitwidth: 16
Expected byte length: 2
```

Figure 33: P4Runtime Controller Initialization Log

As shown in Figure 33, the controller assigns values such as the Cumulative Metadata Label (CML), Random Number (RND), and Mersenne to the ingress edge. The controller log above demonstrates the initialization process, where a CML value of 0 (see chapter 3, section 3.5.3) and a Random Number of 50 are assigned for the PoT mechanism.

PoT Verification Log

```
[19:59:19.531] pot-polka-edge.p4(310) Primitive
[19:59:19.531] last secret = 10
[19:26:55.637] pot-polka-edge.p4(316) Primitive
[19:26:55.637] Verification: 29
[19:26:55.637] pot-polka-edge.p4(319) Primitive
[19:26:55.637] CML_POT = 29
[19:26:55.637] pot-polka-edge.p4(322) meta.verification == meta.cml
is true
[19:26:55.638] pot-polka-edge.p4(324) Primitive
[19:26:55.638] Verification EQUALS CML_POT
[19:26:55.638] pot-polka-edge.p4(325) Primitive
[19:26:55.638] PoT verification successful
```

Figure 34: Log Output Showing Successful PoT Verification

As shown in Figure 34, the verification process successfully confirms that the CML value matches the expected VERIFY value, indicating that the packet traversed the intended path. The SECRET is initially assigned a value of 0 at the ingress edge, but it is reconstructed at each node and reassembled at the egress edge, where it shows a value of 10 (see Chapter 2, Section 2.5.3) for use in the verification equation. As shown, the CML_POT value is equal to 29. Since the RND value is 50, based on the following equation:

$$\text{VERIFY} = (S + \text{RND}) \bmod M$$

the result is calculated as (10 + 50) mod 31 = 29. This equation ensures that the computed VERIFY value matches the expected value by using the SECRET, RND, and the Mersenne prime modulus to prevent tampering.

4.2 Deviation Phase

To simulate a path deviation, we deliberately set an incorrect final CML value (0) before performing the verification check. In a typical scenario, deviations may occur due to packet rerouting, link failures, or intentional attacks that divert packets through unauthorized network paths. By setting the wrong final CML value, we emulate such deviations without modifying the network topology.

This deviation detection capability is crucial for maintaining path integrity in dynamic and potentially adversarial environments. It underscores the robustness of the PoT-PolKA framework in identifying inconsistencies early and preventing security breaches or policy violations. Furthermore, it demonstrates how cumulative metadata-based verification schemes can provide lightweight, scalable security for future programmable networks.

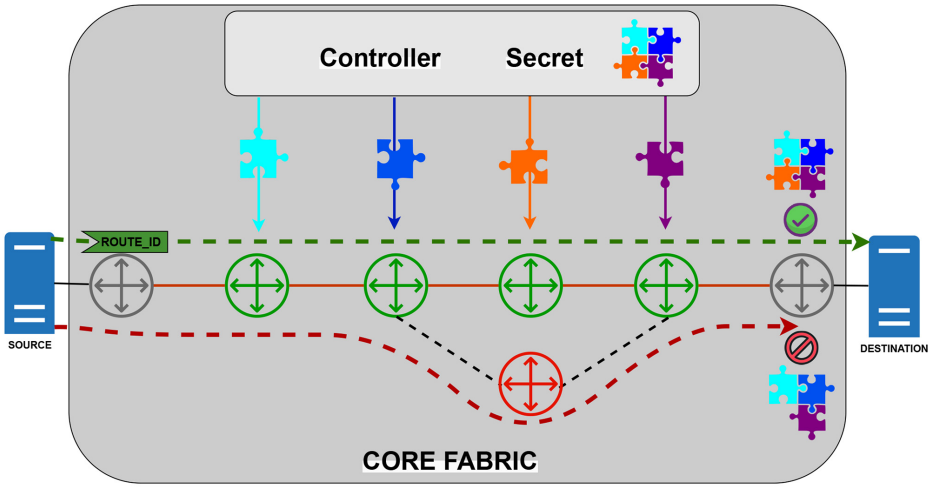


Figure 35: Packet Deviation

As shown in Figure 35, packets initially follow a primary route determined by a designated ROUTE_ID. If the secret information is incomplete or incorrect, packets deviate from the main (green) route and are redirected onto an alternative (red dashed) path. This deviation leads the packet to a dead-end, where it is ultimately dropped. Consequently, unauthorized or improperly configured traffic is prevented from reaching the intended destination, enhancing the overall security and reliability of the network.

PoT Verification Log

```
[18:48:45.688] CML_POT = 0
[18:48:45.688] pot-polka-edge.p4(323) meta.verification == meta.cml
is false
[18:48:45.688] PoT verification failed: incorrect RND or CML values
[18:48:45.688] Action MyIngress.drop
[18:48:45.688] Dropping packet at the end of ingress
```

Figure 36: Log Output Showing Failed PoT Verification

As shown in Figure 36, the verification process detects that the computed CML does not match the expected VERIFY value. This discrepancy indicates that the packet did not follow the intended path. In response, the verification mechanism triggers an immediate drop action to prevent the compromised packet from continuing in the network.

5 Conclusions

This thesis has proposed and demonstrated a novel Proof-of-Transit (PoT) framework, PoT-PolKA, designed to ensure secure and verifiable path compliance in programmable networks. By integrating Shamir's Secret Sharing Scheme (SSSS) with Polynomial Key-based Architecture (PolKA) principles, the system achieves efficient and scalable path verification while minimizing reliance on per-hop state and complex routing tables.

The implementation, based on P4 and P4Runtime, illustrates the feasibility of deploying cryptographic path validation mechanisms within programmable data planes. Through the use of polynomial interpolation, residue number systems, and modular arithmetic optimizations, including Mersenne number-based modulo computations, the proposed design enables lightweight and secure cumulative metadata updates across the network path. Experimental validation within a Mininet-based emulation environment confirms the system's ability to accurately verify packet transit, dynamically adapt to routing changes, and promptly detect unauthorized path deviations.

The separation of control and data planes, achieved via P4Runtime, facilitates dynamic rule installation and policy enforcement without requiring recompilation or service interruption, further demonstrating the flexibility and robustness of the approach.

Future research may focus on extending the architecture to larger and more dynamic network topologies, exploring advanced cryptographic primitives for enhanced security guarantees, and evaluating the system's performance under real-world traffic conditions. Furthermore, the integration of PoT-PolKA mechanisms with inter-domain trust models and blockchain-based verification systems represents a promising avenue for advancing secure and accountable network infrastructures.

In conclusion, the results of this study affirm the potential of programmable networks to deliver scalable, efficient, and secure path verification mechanisms, contributing to the broader objective of resilient and trustworthy next-generation network architectures.

References

[1] F. Brockners, S. Bhandari, T. Mizrahi, S. Dara, and S. Youell. Proof of transit. Internet-Draft draft-ietf-sfc-proof-of-transit-08, Internet Engineering Task Force, Nov 2020. Work in Progress.

[2] Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, and David Walker. P4: Programming protocol-independent packet processors. *SIGCOMM Computer Communication Review*, 44(3):87–95, Jul 2014.

[3] P4Lang. v1model.p4. `https://github.com/p4lang/p4c/blob/master/p4include/v1model.p4`, 2023. Accessed: May 9, 2025.

[4] P4Lang. Behavioral model (bmv2). `https://github.com/p4lang/behavioral-model`, 2023. Accessed: May 9, 2025.

[5] The P4.org API Working Group. P4runtime specification. `https://p4.org/p4-spec/p4runtime/main/P4Runtime-Spec.html`, 07 2021.

[6] C. Dominicini, G. Vassoler, L. Meneses, R. Villaca, M. Ribeiro, and M. Martinello. Polka: Polynomial key-based architecture for source routing in network fabrics. In *2020 6th IEEE Conference on Network Softwarization (NetSoft)*, pages 326–334, 2020.

[7] C. Dominicini et al. Deploying PolKA source routing in P4 switches: (Invited paper). In *Proceedings of the International Conference on Optical Network Design and Modeling (ONDM)*, pages 1–3. IEEE, 2021.

[8] Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, Nov 1979.

[9] B. Tuckerman. The 24th Mersenne prime. *Proceedings of the National Academy of Sciences*, 68(10):2319–2320, 1971.

[10] D. Quadling. Lagrange's interpolation formula. *The Mathematical Gazette*, 50(374):372–375, 1966.

[11] Everson S. Borges, Magnos Martinello, Vitor B. Bonella, Abraão J. dos Santos, Roberta L. Gomes, Cristina K. Dominicini, Rafael S. Guimarães, Gabriel T. Menegueti, Marinho Barcellos, and Marco Ruffini. Pot-polka: Let the edge control the proof-of-transit in path-aware networks. In *Proceedings of the IEEE Conference*, 2023.

[12] Rogério Leão Santos de Oliveira, Christiane Marie Schweitzer, Ailton Akira Shinoda, and Ligia Rodrigues Prete. Using Mininet for Emulation and Prototyping Software-Defined Networks. In *Proceedings of the IEEE International Conference on Communications (ICC)*, pages 1–6, Sydney, Australia, 2014. IEEE. 978-1-4799-4340-1/14/$31.00 ©2014 IEEE.

[13] Rashid Mijumbi, Joan Serrat, Juan-Luis Gorricho, Niels Bouten, Filip De Turck, and Raouf Boutaba. Network function virtualization: State-of-the-art and research challenges. *IEEE Communications Surveys & Tutorials*, 18(1):236–262, 2016. 1st Quarter.